CSE 4113 – Internet Programming Lab

Project Report

Hopper

Teams: Four-Eyed Raven & Byte Me

Submitted By

Team Member	Student ID
Taif Ahmed Turjo	2021-811-221
Fayek Ahmed	2021-511-233
Tanzila Khan	2021-011-201
Md. Muhaiminul Islam Ninad	2021-111-219
Anindya Kundu	2021-911-185
Kabya Mithun Saha	2021-111-192
Tazkia Malik	2021-111-183
Md. Sadek Hossain Asif	2021-211-191

Batch 28

Department of Computer Science & Engineering

University of Dhaka

Submitted On

23.07.2026

Contents

Project Links

Item	URL
Public Git Repository	https://github.com/CREVIOS/Hopper
Deployed Application URL	https://hopper.farefin.com
API Docs (Swagger/Postman) – public link	https://hopper.farefin.com/api/docs
Postman Collection / API Documentation	https://documenter.getpostman.com/view/56891420/2sBY4QtLGS
Demo Video	https://youtu.be/9BCcEsXJXi0

1. Introduction

1.1 Problem Definition & Context

Hopper is a self-hosted compute provisioning platform intended for university environments. The repository describes it as a system that slices institution-owned bare-metal resources into isolated runtime environments that can be requested through a web interface, accessed over SSH, and governed through credit-based usage control and single sign-on integration. In the current implementation, the platform exposes these runtimes as Kubernetes-backed pod sessions while preserving the user-facing concept of provisioned virtual machine style workspaces.

The codebase shows that the project addresses several operational problems at once. First, shared academic compute infrastructure requires stronger isolation than ad hoc account sharing or manually provisioned hosts. Second, institutions need a practical way to manage fair usage, including per-session billing and automatic shutdown when credits are exhausted. Third, a university deployment must integrate with centralized identity management rather than maintaining a separate manual authentication system. Hopper therefore combines a browser-based control plane, an API layer, an orchestration service, a relational ledger-backed data model, and Keycloak-based authentication in one deployable platform.

1.2 Target Users & Use Cases

Hopper serves three distinct user groups whose responsibilities are enforced through Keycloak roles and mirrored in both frontend route guards and backend authorization checks. Rather than treating every authenticated user as a generic cluster consumer, the platform separates runtime consumption, academic resource distribution, and infrastructure governance into clearly bounded workflows. This role split is visible in the implemented routes: students mainly operate within the dashboard, pod, credit-history, and SSH settings screens; professors work through the teaching console to fund students; and administrators manage the wider platform from the admin console.

Role-Oriented Product Surface

Student users consume compute time and interact directly with provisioned environments. **Professor** users act as academic resource managers who distribute credits to learners. **Admin** users supervise the platform itself, including identity, approvals, capacity visibility, and system-wide funding decisions. The result is a product surface that aligns with how a university lab actually operates: request, fund, supervise.

The visual split above highlights the system's central design decision: Hopper is not only a VM launcher for students, but also a teaching-budget workflow for faculty and an operations console for administrators. That separation makes the platform suitable for institutional deployment, where fairness, approval, and observability are as important as

Table 1: Primary target users and confirmed Hopper use cases

User Group	Primary Goal	Main Interfaces	Confirmed Use Cases
Student	Access course compute environments on demand	Dashboard, Pods, Pod Detail, Credits, SSH Settings	Sign up and sign in, launch a VM from a plan, wait in the provisioning queue when capacity is full, open terminal and file access, connect over SSH, monitor usage and remaining credits, terminate sessions, and report runtime issues for admin follow-up.
Professor	Distribute teaching resources to students fairly	Teacher Console, Credits API, user-authenticated dashboard flows	Review available teaching balance, inspect student balances, allocate credits to individual students, and manage course support without requiring direct infrastructure access.
Admin	Govern users, budgets, and cluster operations	Admin Console, approval flows, audit and platform views	Approve pending teacher requests, grant credits, inspect users and roles, review active VMs, monitor node and platform statistics, inspect audit information, and resolve platform-level issues raised by users.

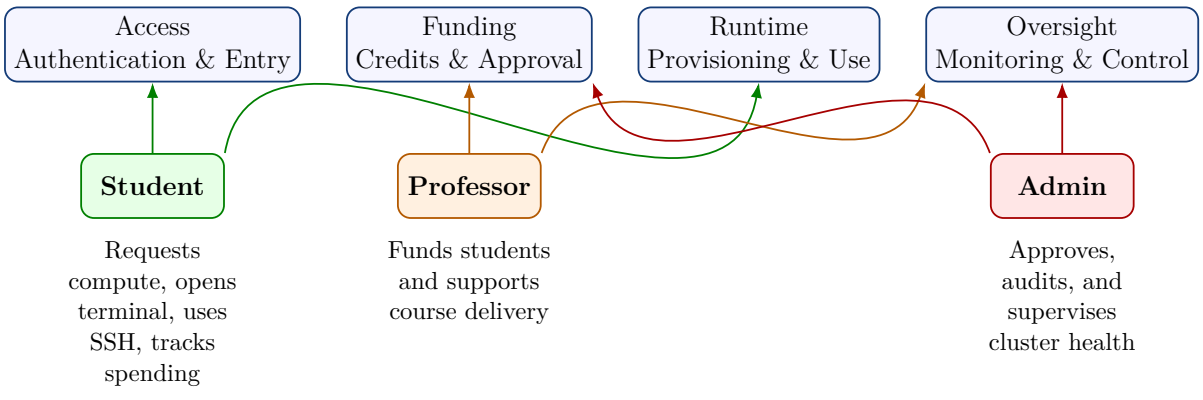

Figure 1: How Hopper's three user groups map onto the platform lifecycle

raw compute access.

1.3 Core Features

Hopper is a multi-service compute-management platform that combines federated authentication, admission-controlled resource provisioning, metered billing against a double-entry ledger, real-time monitoring, and cluster-based workload orchestration. The features summarised below were confirmed against the implementation rather than against planning documents; where a capability is only partially implemented, the limitation is stated explicitly.

Two properties distinguish the system from a thin wrapper over the Kubernetes API. First, provisioning is admission-controlled: a request is accepted only after both a credit-balance check and a cluster-capacity check succeed, and a request that passes the former but fails the latter is queued rather than rejected. Second, billing is enforced end to end without operator intervention—the orchestrator meters running sessions, the gateway debits a double-entry ledger, and exhaustion of a balance propagates back to the orchestrator as a termination signal.

Table 2: Summary of Hopper's core features

Category	Features
Identity and access control	Keycloak-backed OpenID Connect sign-in using the authorization-code flow with PKCE, covering callback handling, token refresh, and logout with token revocation; JSON Web Key Set validation of access tokens with issuer verification; HTTP-only session cookies; a student, professor, and admin role hierarchy derived from realm roles; API keys for programmatic access alongside interactive sign-in; and email verification and code-based password reset.
Compute provisioning and orchestration	Plan-based session creation across small, medium, and large tiers with fixed CPU, memory, disk, and hourly-credit definitions; credit-balance validation performed before any cluster operation is attempted; Kubernetes pod lifecycle management in a Go service exposed over gRPC, covering creation, termination, status queries, node listing, and metric streaming; an informer-driven state machine that reconciles observed pod state; per-pod SSH services with generated credentials; and selectable language images for Python, Java, and C++ workloads.

Category	Features
Admission control and scheduling	A capacity-aware scheduler that admits a request only when cluster resources allow, and otherwise enqueues it; a persistent queue exposing position and depth to the requesting user; a reconciliation pass that requeues requests stuck in the admitting state; and database-backed leader election ensuring a single active scheduler across replicas, with independent leader election on the orchestrator side.
Credits, billing, and accounting	A double-entry credit ledger in which every transfer writes balanced debit and credit entries with a running balance snapshot; per-minute metering of running sessions published by the orchestrator over NATS; idempotent consumption of billing events in the gateway, keyed by transaction identifier; low-balance warning and balance-exhaustion events, the latter propagating back to the orchestrator to terminate the affected session; and professor-to-student credit allocation governed by a request-and-approval workflow.
User-facing workspace capabilities	Real-time CPU and memory metrics delivered over Server-Sent Events; a browser-based terminal backed by a WebSocket pseudo-terminal session; a browser-based editor served through an authenticated code-server proxy with per-user settings persistence; file browsing, upload, and download over SFTP, with path-traversal protection (deletion and renaming are not currently exposed); SSH public-key management with fingerprint computation; a notification feed with unread counts delivered over a second Server-Sent Events stream; and an issue-reporting workflow with administrative resolution.
Administration and oversight	Administrative dashboards covering user management, role assignment, course records, cluster node inspection, platform statistics, and active-session listings; a separate professor dashboard for cohort oversight and credit allocation; a professor-role request and approval workflow; request-level audit logging with an administrative viewer; and historical usage and spend analytics backed by a TimescaleDB hypertable and rendered as usage and spend charts.
Operations, reliability, and security	A session reaper that terminates expired or credit-exhausted sessions automatically; per-session network isolation groups compiled into Cilium network policies; request rate limiting at the gateway; a Prometheus, Grafana, and Loki observability stack with alerting rules; and a test suite spanning unit, integration, end-to-end, load, security, and chaos scenarios.

Category	Features
Supporting infrastructure	SvelteKit frontend, FastAPI gateway, Go orchestration service, Kubernetes workloads, PostgreSQL with TimescaleDB, NATS JetStream, Keycloak, Nginx Ingress, and cert-manager-issued TLS; deployment expressed declaratively as a Helm chart with Argo CD applications, alongside Pulumi and Ansible definitions for infrastructure provisioning.

1.4 Verified Feature Inventory

For completeness, the repository was also audited feature-by-feature across the frontend routes, API routers, service modules, and tests. The following inventory is ordered intentionally: the major product capabilities appear first, followed by the secondary or supporting capabilities that complete the current application.

Major product capabilities

1. **VM provisioning and lifecycle management**: launch VM-style pod sessions from the web UI; choose a plan (`small`, `medium`, `large`); choose a base image/template (Ubuntu, Python, C/C++, Java); view session lists and detail pages; and terminate running sessions.
2. **Interactive development access**: connect over SSH, open an in-browser terminal backed by a pseudo-terminal WebSocket, and open a browser-based VS Code/code-server workspace through the authenticated backend proxy.
3. **Queue-based admission control**: when cluster capacity is unavailable, launch requests are queued rather than rejected; users can view queue position and queue depth, queued sessions start automatically as capacity frees up, and still-queued requests can be cancelled.
4. **Credit-based billing**: the platform maintains per-user balances, deducts usage charges while sessions run, stops billing when a session stops, exposes transaction history and spend analytics, warns users about low balances, estimates remaining runtime from current burn rate, and terminates sessions automatically when credits are exhausted.
5. **Usage monitoring**: the dashboard and pod pages expose real-time and historical CPU and memory metrics, per-pod usage charts, per-user aggregated usage summaries, recent activity summaries, and plan-distribution or spend visualizations.
6. **Role-based operation**: the application distinguishes student, professor, and admin roles and exposes different routes, permissions, and workflows to each role.
7. **Authentication and account management**: local email/password sign-in,

Keycloak-backed OIDC sign-in, signup, email verification, password reset, token refresh, logout, and current-user session introspection are all implemented.

8. **Teacher workflow**: professors can open a dedicated teaching console, inspect searchable student lists, view student balances, and allocate credits from their own funded budget to students.
9. **Administrative workflow**: administrators can inspect platform statistics, active VMs, cluster nodes, users, teacher-approval requests, issue reports, audit logs, and role-management controls, and they can allocate credits to users.

Remaining confirmed capabilities

- **Provisioning constraints**: a user can run at most three concurrent VMs; cluster availability is exposed as free CPU, memory, storage, ready-node count, and queue length.
- **Network-aware placement**: `network_group` placement exists and is restricted to professor or admin initiated sessions.
- **Dashboard UX**: the student dashboard includes active-VM counts, credit balance, average CPU and memory summaries, plan-distribution charts, recent VM activity, recent credit activity, and low-balance banners.
- **Queue UX**: the queue page refreshes positions over time, distinguishes queued from admitting entries, and exposes cancel controls only while cancellation is still valid.
- **Workspace detail UX**: the pod-detail page exposes overview, terminal, and files tabs; access details such as SSH port and password; a VS Code launcher; live metrics; billing panels; and historical usage charts.
- **File management**: users can browse directories inside a running VM, upload files, and download files, with path-safety validation and ownership enforcement.
- **SSH-key management**: users can add, list, and delete SSH public keys; fingerprints are computed and displayed; duplicate keys are rejected; and a per-user limit of ten keys is enforced.
- **Notification center**: users receive a notification feed with unread counts, single-item and mark-all-as-read actions, and live Server-Sent Event delivery.
- **Issue reporting**: users can report problems, optionally link them to a pod session, review their own submitted issues, and administrators can review and resolve those issues.
- **API keys**: users can create, list, and revoke API keys for programmatic access, with role-sensitive restrictions retained at the API layer.
- **Teacher approval**: signups requesting the teacher role are created as pending requests, then approved or rejected by administrators.

- **Admin safeguards**: the admin role editor prevents invalid role assignments, self-demotion, and demotion of the last remaining administrator.
- **Account recovery**: the application supports email-code verification, resend-code flows, forgot-password initiation, and password reset confirmation.
- **Landing and onboarding pages**: the frontend includes a public landing page, login page, signup page, email-verification page, forgot-password page, and authenticated dashboards.
- **Security controls**: domain-restricted signup is supported, password-policy checks are mirrored before Keycloak submission, rate limiting is applied to authentication and VM-creation flows, and secure cookie handling is used for browser sessions.
- **Observability and operations**: the repository includes Prometheus, Grafana, Loki, audit logging, health-oriented operational endpoints, and broad unit, integration, end-to-end, load, chaos, and security testing coverage.
- **Developer/test-only helper**: a dedicated development login route exists for local or automated test workflows and is not part of the normal production user flow.

1.5 Tech Stack Overview

The frontend uses SvelteKit 2, Svelte 5, TypeScript, and Vite to support server-rendered routes, interactive dashboards, and reusable authenticated components. State is managed through Svelte writable stores, derived state, and route-level server load functions.

The backend uses FastAPI with Python 3.12, Pydantic, SQLAlchemy, Alembic, SSE, rate limiting, and asynchronous I/O. This provides typed APIs, automatic OpenAPI documentation, validation, and clear separation of routers, services, models, and middleware. A Go 1.23 orchestration service uses gRPC and Kubernetes libraries for compute-session lifecycle management.

PostgreSQL with TimescaleDB stores relational and time-series data. NATS JetStream handles asynchronous billing, metrics, and lifecycle events, while Keycloak provides OIDC-based authentication and role-based access control.

Deployment is based on Kubernetes, Nginx Ingress, Helm, and infrastructure files under `k8s/`, `charts/`, and `infrastructure/`. Docker Compose supports local development, cert-manager automates TLS certificates, Cilium enforces network policies, and optional email delivery is configured through Brevo or SMTP.

Table 3: Summary of the Hopper technology stack

Category	Technologies	Purpose and suitability
Frontend	SvelteKit 2, Svelte 5, TypeScript, Vite	Provides server-side rendering, route-based application structure, interactive dashboards, reusable components, and type-safe frontend development.
State management	Svelte writable stores, derived state, server load functions	Manages authentication, pod state, notifications, and route-level data without requiring a heavyweight external state-management library.
API backend	FastAPI, Python 3.12, Pydantic, SQLAlchemy, Alembic	Supports typed asynchronous APIs, validation, database access, migrations, middleware, and automatic OpenAPI documentation.
Orchestration service	Go 1.23, gRPC, Kubernetes client libraries	Handles cluster-facing pod lifecycle operations using mature native Kubernetes tooling and efficient service-to-service communication.
Database	PostgreSQL, TimescaleDB	Stores relational application data while providing time-series capabilities for infrastructure and usage metrics.
Messaging	NATS JetStream	Enables asynchronous distribution of lifecycle, billing, notification, and metrics events between services.
Authentication	Keycloak, OpenID Connect	Provides centralized authentication, institutional sign-on support, token validation, and role-based access control.
Deployment	Kubernetes, Nginx Ingress, Helm, cert-manager	Supports workload provisioning, service discovery, ingress routing, declarative deployment, and automated TLS certificate management.
Security infrastructure	Cilium network policies, environment variables, Kubernetes Secrets	Restricts service communication, protects credentials, and provides namespace-level network isolation.
Local development	Docker, Docker Compose	Provides reproducible local instances of PostgreSQL, TimescaleDB, NATS, Keycloak, and supporting services.
Email service	Brevo or SMTP	Supports verification, password-reset, and notification emails, with production configuration supplied through environment variables.

2. System Architecture

2.1 High-Level Architecture Diagram

The deployed system is organized as a browser-facing frontend, a Python API gateway, a Go orchestration service, an identity provider, a relational database layer, and an event bus. The ingress configuration under `k8s/deploy/03-ingress.yaml` shows that external traffic is split across four path families: `/` for the frontend, `/api` for the gateway, Keycloak realm/resource paths for authentication, and a dedicated code-server proxy path for browser-based development access. The API gateway then coordinates synchronous database work, gRPC calls to the orchestrator, and asynchronous event handling over NATS.

At runtime, the orchestrator is the component that directly manages Kubernetes resources. The FastAPI gateway does not create pods itself; instead it creates or updates the application-facing database state, invokes the orchestrator over gRPC, and consumes follow-up events such as billing, pod lifecycle changes, and metrics updates. PostgreSQL stores the authoritative relational state for users, sessions, the credit ledger, notifications, queue entries, and other application records, while TimescaleDB features are used where available for metrics retention and hypertable support.

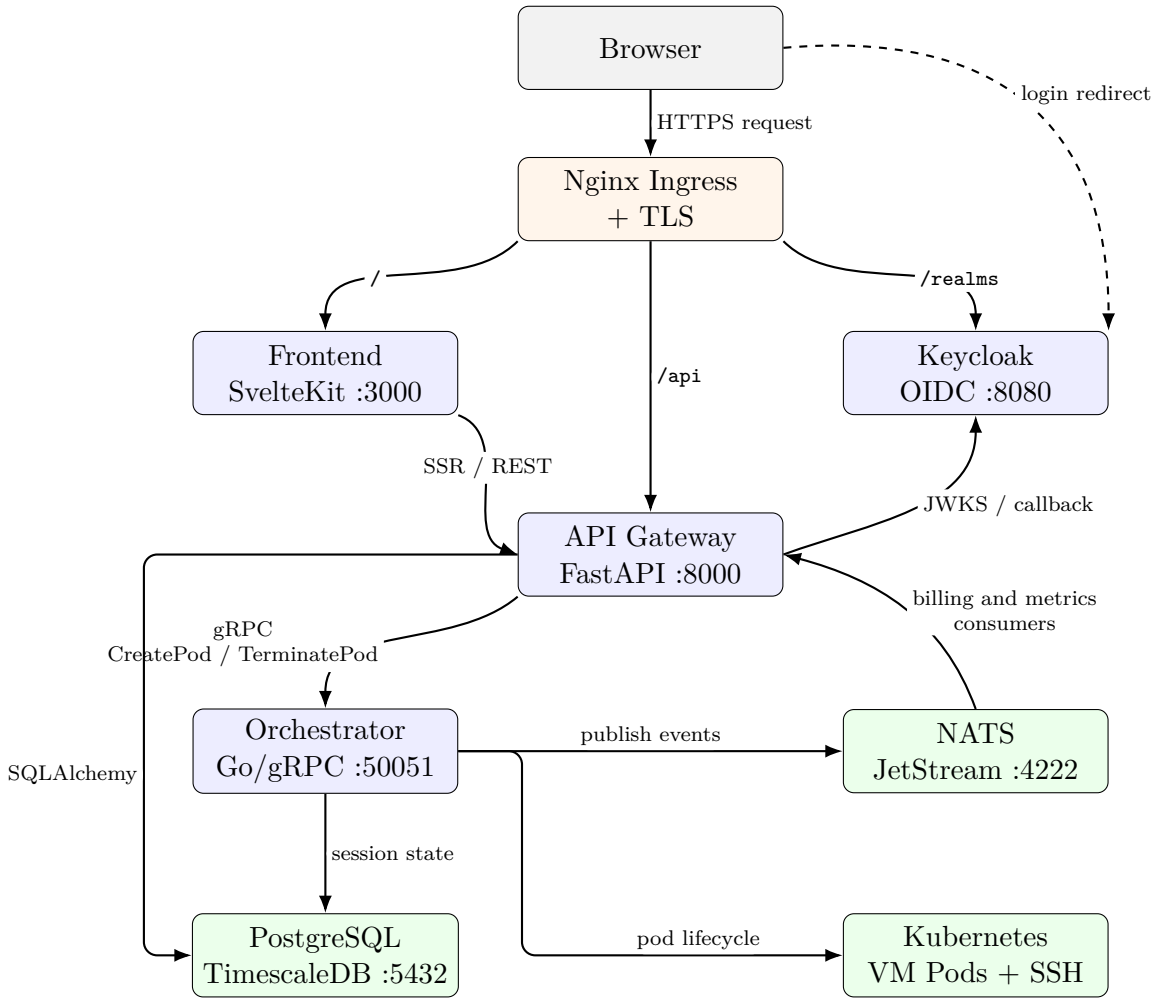

Figure 2: Verified high-level Hopper architecture derived from the frontend, gateway, orchestrator, and Kubernetes deployment files.

Figure 2 reflects the concrete implementation rather than an aspirational design. In particular, the current repository confirms the following communication paths: browser-to-frontend delivery over ingress, frontend-to-gateway HTTP calls, gateway-to-orchestrator gRPC calls, orchestrator-to-Kubernetes lifecycle control, orchestrator-to-NATS event publication, gateway-side NATS consumers for billing and metrics, and Keycloak-mediated authentication with cookie-backed application sessions.

The main operational flow shown in Figure 3 is also verified directly in code. The frontend issues requests through the `/api` path, the gateway checks identity and credits, the scheduler either reserves capacity or enqueues the request, the orchestrator creates or terminates the Kubernetes workload, and asynchronous follow-up state reaches the UI through SSE streams, notification events, and interactive proxies.

2.2 Frontend Architecture

The frontend is a SvelteKit application organized around file-based routes in `frontend/src/routes`. Route-specific server load functions such as `+layout.server.ts`,

`dashboard/+page.server.ts`, `admin/+page.server.ts`, and `pods/[id]/+page.server.ts` fetch authenticated data on the server side before rendering pages. The helper `frontend/src/lib/api/server.ts` resolves whether those server-side requests should go through an ingress-relative `/api` path or directly to an internal API base URL, depending on environment configuration.

Application-wide authentication state is coordinated in `+layout.server.ts` and `+layout.svelte`. The layout loader attempts `/auth/me` using the `session_token` cookie, falls back to `/auth/refresh` when necessary, and returns the authenticated user and balance to the shell layout. The authenticated shell then renders the sidebar, topbar, notification bell, and balance badge, and it also starts periodic refresh and balance polling timers in the browser. This indicates a mixed SSR-plus-client-refresh strategy rather than a purely client-side single-page architecture.

State management remains lightweight and is implemented with built-in Svelte mechanisms instead of an external library such as Redux or Zustand. The repository defines writable stores in `frontend/src/lib/stores/` for authentication, pod-related state, and notifications. Route components also use local derived state extensively, as seen in multiple `+page.svelte` files. This means the application follows a layered state strategy: server-loaded page data for initial render, writable stores for cross-component reactive state, and local derived values for view-level calculations.

The API layer is split between server-side and browser-side helpers. On the browser side, `frontend/src/lib/api/client.ts` wraps `fetch`, automatically includes credentials, retries once after a silent refresh on non-login `401` responses, and exposes a small SSE helper. On the server side, the route loaders call the backend directly through `apiUrl()`. This division is suitable for a SvelteKit application because it allows SSR data fetching, cookie forwarding, and browser reactivity without duplicating request logic in every route.

The real-time and interactive frontend features are implemented explicitly rather than implicitly. The notification store opens an EventSource connection to `/api/notifications/stream`, the pod detail page opens an EventSource to `/api/pods/{id}/metrics`, and the terminal component opens a WebSocket to `/api/pods/{id}/terminal`. Browser-based editor access is routed through the backend proxy path rather than by direct pod exposure. The repository therefore confirms three distinct frontend interaction modes: SSR page loading, credentialed REST-style API calls, and long-lived SSE or WebSocket channels.

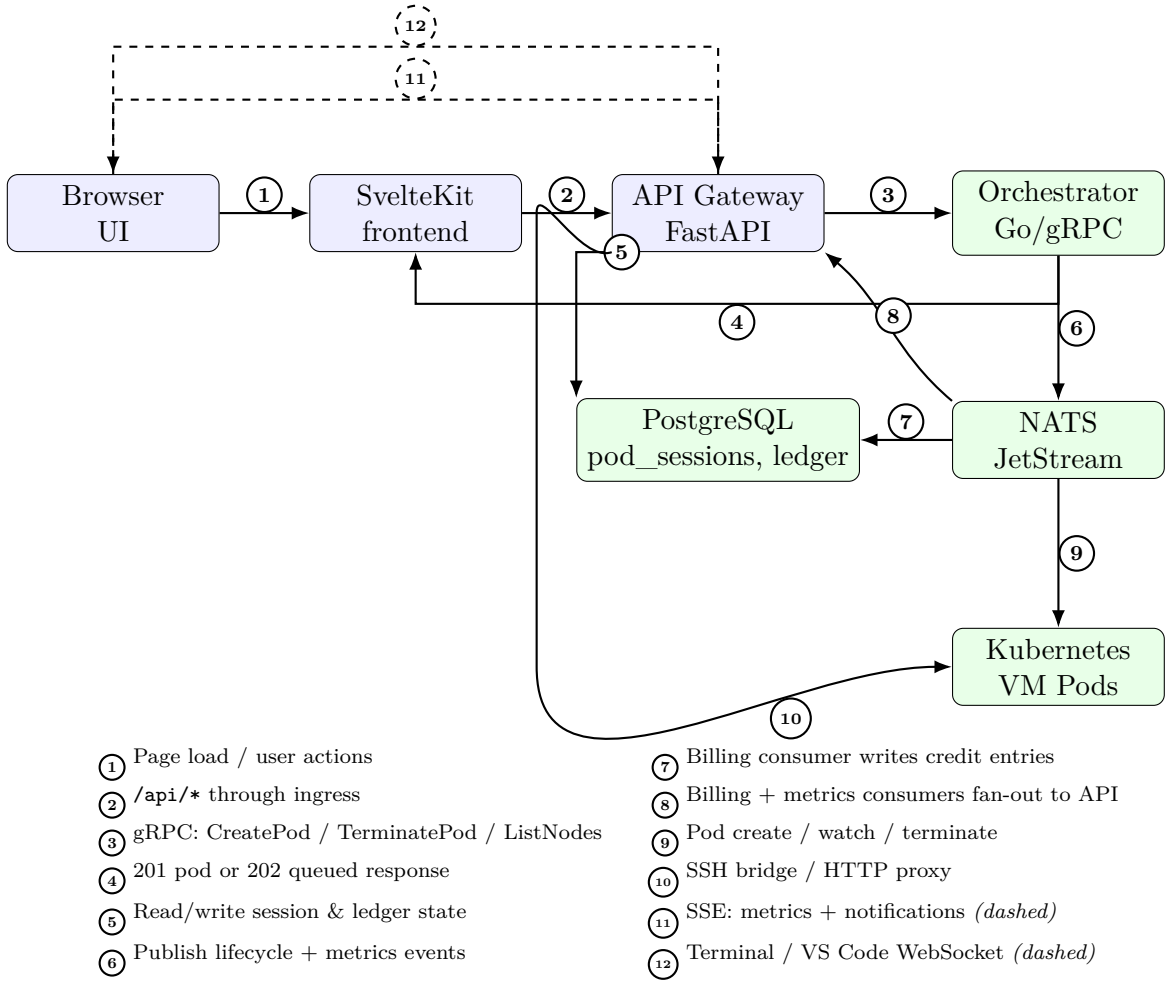

Figure 3: Main request and data flow for VM creation, monitoring, and interactive access.

2.3 Backend Architecture

The backend is implemented as a FastAPI-based API gateway in `services/api-gateway/app/main.py`. Its router registration shows a domain-oriented separation by capability: authentication, pods, credits, admin, settings, SSH keys, files, issues, notifications, and usage each live in their own router modules. Shared concerns are applied through dependency injection and middleware, including database sessions from `get_db()`, authentication from `get_current_user()`, audit logging, CORS control, and rate limiting. This structure is closer to a thin-router plus service-module design than to a heavy layered controller-service-repository stack.

The gateway is not a passive CRUD layer. It coordinates synchronous HTTP work with asynchronous background processes and remote orchestration. The `pods` router, for example, validates access, checks user credits, enforces per-user limits, invokes admission logic in `vm_scheduler` and `vm_queue`, persists `PodSession` state, and then calls the Go orchestrator through the async gRPC wrapper in `orchestrator_client.py`. If synchronous admission is not possible, the same router returns a queued response instead of failing immediately. This demonstrates an application-service style workflow centered

in router handlers and helper service modules.

Several backend services are event-driven. During application startup, the gateway connects to NATS, starts the billing consumer, and starts the metrics consumer. The billing consumer processes `billing.deducted` and `billing.exhausted` events, applies idempotent credit deductions, manages grace-period behavior, and triggers user notifications. The metrics consumer stores periodic metrics snapshots in the database. The notification service persists rolling user notifications and also subscribes to pod lifecycle events so that `pod.started`, `pod.stopped`, and `pod.failed` can be reflected in the user interface. The result is a hybrid backend architecture combining request-response APIs, background event processing, and stateful session updates.

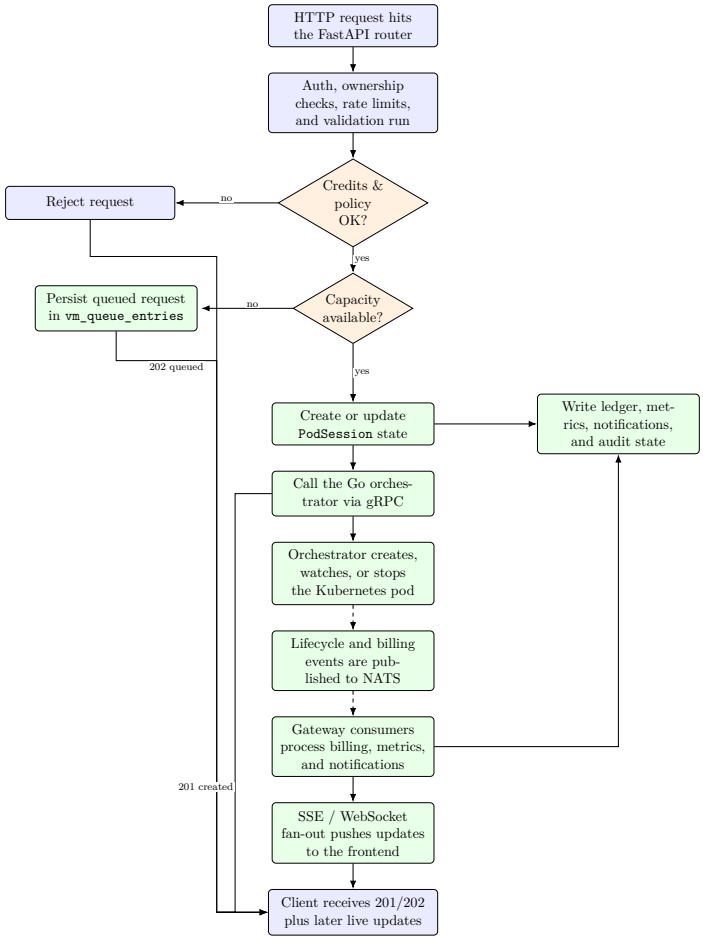

Figure 4: Backend architecture flow: synchronous admission, queued fallback, gRPC orchestration, and asynchronous event fan-out.

The admission queue is an especially important architectural feature. The service `vm_scheduler.py` elects a single leader through the `scheduler_leader` table and runs a first-come, first-served admission loop that considers cluster capacity computed from orchestrator node data plus current `PodSession` reservations. The queue service persists queued requests in `vm_queue_entries`. This means queueing is implemented inside the application database rather than delegated to an external job queue. The code explicitly documents that the loop is head-of-line and does not backfill smaller requests behind a

blocked older request.

The Go orchestration service is separated into focused internal packages under `services/orchestrator/internal/`. The inspected file tree confirms packages for billing, gRPC endpoints, event publication, leader election, Kubernetes access, watchers, and pod management. The main process connects to NATS, creates Kubernetes clients, starts the gRPC server, and then runs singleton work that subscribes to events, reconciles watcher state, and publishes metrics. In other words, the orchestrator is the cluster-facing runtime component, whereas the FastAPI application remains the externally facing control plane.

The current backend does not implement a dedicated repository abstraction layer. Database queries are performed directly with SQLAlchemy `AsyncSession` from routers and service modules. This is an important verified characteristic of the codebase and should not be misrepresented as a repository-pattern implementation. If a stricter persistence abstraction is desired later, that would be future work rather than a description of the current system.

2.4 Database Design

The database design combines normalized relational entities for identity, sessions, credits, and operational metadata with a time-series metrics table that can be promoted to a TimescaleDB hypertable when the extension is available. The foundational migration creates separate tables for users, accounts, transfers, ledger entries, pod sessions, and metrics. Later migrations adapt the original GPU-oriented session model into plan-based VM sessions, add notification and queueing support, and introduce API key and network-group support.

Figure 5 highlights an important characteristic of the current schema: not every logical relationship is enforced with a foreign key. The strongest database-enforced relationships are in the credit ledger, where `ledger_entries.transfer_id` references `transfers.id`, `ledger_entries.account_id` references `accounts.id`, and `vm_queue_entries.pod_session_id` references `pod_sessions.id`. By contrast, many user-owned records such as `pod_sessions`, `notifications`, and `api_keys` store a `user_id` without a declared SQL foreign key. Ownership is therefore enforced primarily through application logic and authenticated route checks rather than through a dense relational constraint graph.

The credit subsystem is intentionally normalized into three tables: `accounts`, `transfers`, and `ledger_entries`. This supports double-entry accounting and avoids storing mutable balances as the only source of truth. The initial migration also creates indexes on `ledger_entries.account_id` and `ledger_entries.transfer_id` and uses PostgreSQL rules to prevent updates or deletes on ledger rows and transfers. That design decision is consistent with the service code, which treats the ledger as append-only and uses transaction-level locking to serialize deductions.

The VM lifecycle subsystem centers on `pod_sessions`. Each row stores the user-facing

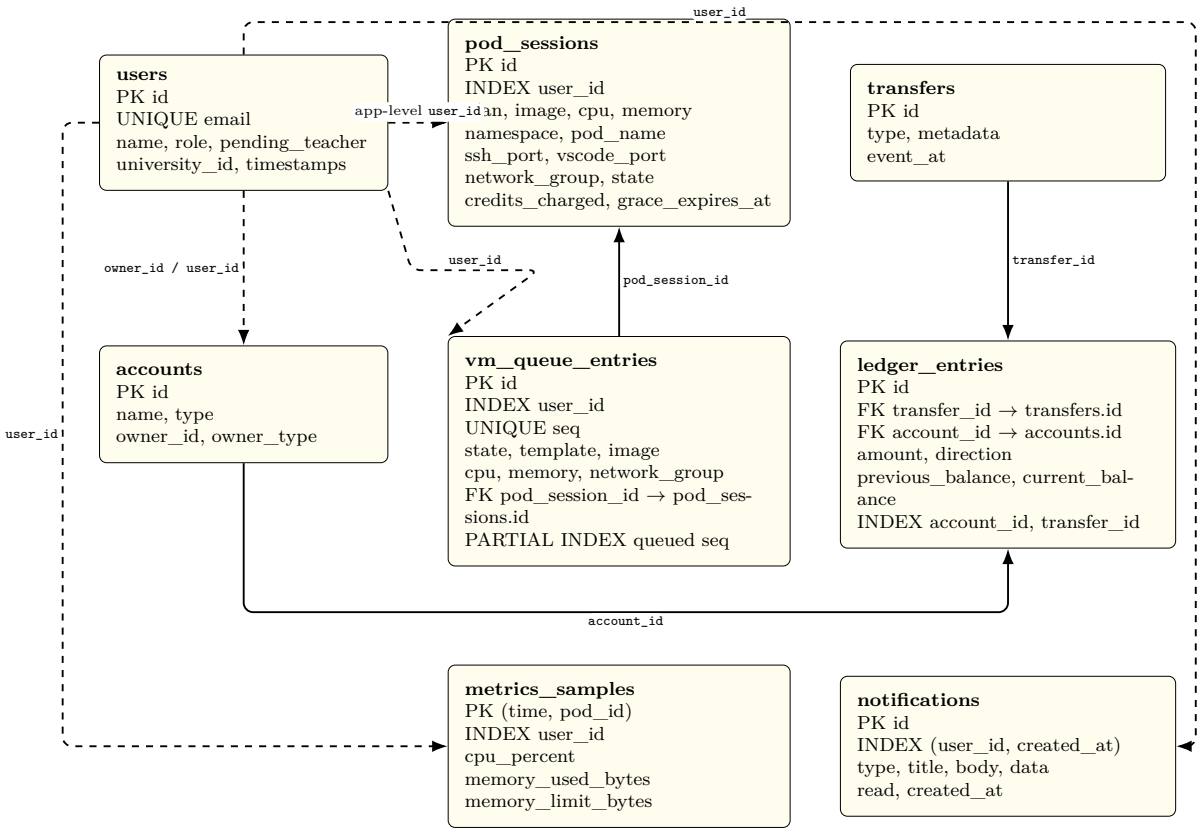

Figure 5: Core entity relationships derived from SQLAlchemy models and Alembic migrations. Solid arrows indicate database-enforced foreign keys; dashed arrows indicate application-level links stored by identifier only.

VM identity, selected plan, image, CPU and memory limits, namespace and pod name, exposed ports, current state, accumulated credits, and optional network-group data. Queueing is separated into `vm_queue_entries`, which freezes the requested plan and resolved image at enqueue time and adds a monotonically increasing `seq` field. The migration creating this table also defines a partial index on queued rows by sequence, which supports efficient first-come, first-served head-of-line admission scans.

The notification and metrics subsystems are also specialized rather than overloaded into a generic event table. Notifications form a rolling per-user window backed by a composite index on `(user_id, created_at)` because the main queries are recent notifications and unread counts. Metrics are stored in `metrics_samples` with a composite primary key `(time, pod_id)` and a separate index on `user_id`. The migration attempts to create a Timescale hypertable and retention policy when the database supports those functions, but it explicitly degrades gracefully to a regular PostgreSQL table when TimescaleDB is unavailable.

Several auxiliary tables exist outside the core ERD and contribute to operational completeness: `audit_logs`, `ssh_keys`, `issue_reports`, `email_codes`, `scheduler_leader`, and `api_keys`. These support auditing, SSH management, issue reporting, email verification and reset flows, leader election for the scheduler loop, and scoped programmatic

access. Because Section 2 focuses on the main architectural data paths, they are described here textually rather than all being included in one overloaded ERD figure.

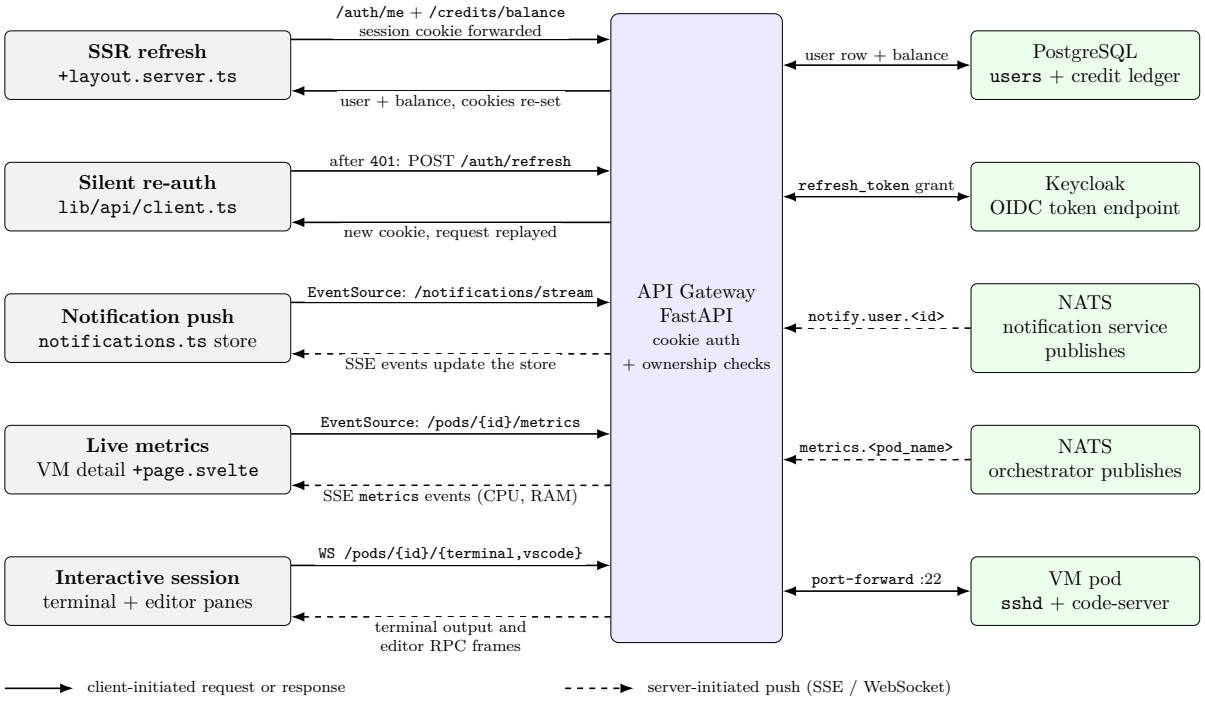

Figure 6: Verified automatic UI data-sync mechanisms: SSR refresh, SSE streams, and WebSocket proxying.

The repository confirms that automatic UI synchronization is implemented in multiple ways rather than through one generic push channel. As shown in Figure 6, the layout loader refreshes authentication state and forwards cookies server-side, the browser-side API client retries after silent refresh on eligible `401` responses, notifications are pushed over an `EventSource` stream, pod metrics are streamed through a separate `EventSource`, and terminal or browser-based editor access is proxied through WebSocket or HTTP upgrade paths in the gateway. This is a concrete, code-backed real-time strategy. It should therefore be documented as a combination of SSR data refresh, SSE fan-out, and backend-mediated interactive proxies rather than as a generic polling-only or WebSocket-only system.

3. API Documentation

3.1 API Design Overview

Hopper exposes a primarily REST-style HTTP API through the FastAPI gateway in `services/api-gateway/app/main.py`. The frontend API client hard-codes `/api` as its public base path, and the Kubernetes ingress rewrites that path before forwarding requests to the gateway service. Internally, the application routers are mounted directly at prefixes such as `/auth`, `/pods`, `/credits`, and `/admin`; externally, users reach them as `/api/auth`, `/api/pods`, `/api/credits`, and so on. The application metadata declares

API version `0.1.0`, but the repository does not implement URI-based versioning such as `/v1`. The current versioning approach is therefore documentation-level rather than path-level.

The dominant request and response format is JSON over HTTPS. FastAPI and Pydantic models are used for request validation and typed response shaping, for example `LoginRequest`, `SignupRequest`, `CreatePodRequest`, `PodResponse`, `CreditBalanceResponse`, and `UserResponse`. The frontend client sets `Content-Type: application/json` automatically for non-form submissions and includes browser credentials on every request. A smaller set of endpoints uses alternative transports where the feature requires them: file upload uses `multipart/form-data`, file download returns `application/octet-stream`, notification and metrics feeds use Server-Sent Events (SSE), and terminal or browser-editor sessions use proxied WebSocket or HTTP upgrade paths.

Authentication is enforced through the shared dependency `get_current_user()`. The main browser flow uses secure HttpOnly cookies, especially `session_token` and `refresh_token`, which are issued after successful Keycloak-backed sign-in. Programmatic access is also supported through Hopper API keys supplied either in `X-API-Key` or as `Authorization: Bearer hop_...`. The code explicitly restricts `read_only` API keys to `GET`, `HEAD`, and `OPTIONS` requests, while API-key creation and revocation require a real cookie-backed session. Authorization is role-aware rather than endpoint-global: the repository verifies at least three roles, namely `student`, `professor`, and `admin`.

Validation and error handling are a combination of schema validation and route-level checks. Pydantic enforces field structure, lengths, and constrained values, while the routers enforce application rules such as credit sufficiency, ownership, allowed domains, password policy, and role restrictions. The common error format is FastAPI's JSON `{"detail": "..."}` payload, although validation failures inherited from framework-level schema errors may return the standard structured `422` detail list. The codebase also shows consistent use of meaningful HTTP status codes such as `201 Created`, `202 Accepted`, `204 No Content`, `401 Unauthorized`, `402 Payment Required`, `403 Forbidden`, `404 Not Found`, `409 Conflict`, `422 Unprocessable Entity`, and `429 Too Many Requests`.

The API does not follow one universal pagination contract, but several repeatable conventions are implemented. Credit history and audit-log style listings use `limit` and `offset` query parameters. Notification and issue listing endpoints impose bounded maximums, and issue administration accepts a simple `status` filter. Usage endpoints accept a `range` parameter with confirmed values `1h`, `6h`, `24h`, and `7d` to control aggregation windows. These conventions are pragmatic and code-backed, but they are not wrapped in a separate global pagination abstraction at the current stage of the project.

3.2 Endpoint Reference

The report focuses on the important public application endpoints that are intended for user-facing or evaluator-facing interaction. Internal readiness and liveness handlers such as `/healthz` and `/readyz` are deliberately excluded from the main table because they are deployment-facing operational probes rather than end-user APIs.

Authentication and identity endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/auth/login`	Starts the OIDC login flow with PKCE, state, and nonce, then redirects the browser to Keycloak.	No	Public
POST	`/api/auth/signup`	Creates a local-account sign-up request with role selection and password-policy checks.	No	Public
POST	`/api/auth/verify-email`	Confirms an emailed verification code.	No	Public
POST	`/api/auth/resend-code`	Re-sends a verification or password-reset code.	No	Public
POST	`/api/auth/forgot-password`	Starts the password-reset flow for an existing account email.	No	Public
POST	`/api/auth/reset-password`	Resets a password after code validation and, where applicable, marks the email verified.	No	Public
POST	`/api/auth/login`	Performs direct email-password login and issues session cookies on success.	No	Public
GET	`/api/auth/callback`	Completes the OIDC authorization-code exchange after Keycloak redirects back.	No	Public callback endpoint
POST	`/api/auth/refresh`	Uses the refresh-token cookie to mint a new access cookie.	Refresh cookie	Session holder
GET	`/api/auth/me`	Returns the authenticated user's current profile and pending-teacher status.	Yes	Any authenticated user
POST	`/api/auth/logout`	Revokes the refresh token where possible, clears cookies, and redirects to Keycloak logout.	Yes	Any authenticated user
POST	`/api/auth/api-keys`	Creates an API key and returns the plaintext key exactly once.	Yes	Any authenticated user with session cookie
GET	`/api/auth/api-keys`	Lists the caller's API keys without revealing stored secrets.	Yes	Any authenticated user
DELETE	`/api/auth/api-keys/\{key\_id\}`	Revokes one of the caller's API keys.	Yes	Any authenticated user with session cookie

Pod lifecycle and interactive session endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/pods/plans`	Returns the plan catalogue exposed to the provisioning UI.	No	Public
GET	`/api/pods/`	Lists the current user's pod sessions.	Yes	Any authenticated user
POST	`/api/pods/`	Creates a pod synchronously when capacity is available, or returns `202 Accepted` with queue information when the request is enqueued.	Yes	Any authenticated user; `network_group` limited to professor/admin
GET	`/api/pods/availability`	Reports whether synchronous capacity is currently available together with queue length.	Yes	Any authenticated user
GET	`/api/pods/queue`	Lists the caller's active queue entries and queue positions.	Yes	Any authenticated user
DELETE	`/api/pods/queue/\{entry\_id\}`	Cancels one of the caller's still-queued VM requests.	Yes	Queue-entry owner
GET	`/api/pods/\{pod\_id\}`	Returns details of a specific pod session.	Yes	Pod owner
DELETE	`/api/pods/\{pod\_id\}`	Terminates a pod session and releases its reserved resources.	Yes	Pod owner
GET	`/api/pods/\{pod\_id\}/metrics`	Streams live pod metrics to the UI through Server-Sent Events.	Yes	Pod owner
HTTP proxy	`/api/pods/\{pod\_id\}/vscode/\{path\}`	Proxies browser-based code-server traffic for an owned pod across `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, and `OPTIONS`.	Yes	Pod owner
WS	`/api/pods/\{pod\_id\}/vscode/\{path\}`	WebSocket upgrade path used by the proxied browser editor.	Yes	Pod owner
WS	`/api/pods/\{pod\_id\}/terminal`	Provides interactive terminal access to a running pod.	Yes	Pod owner

Credits and usage endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/credits/balance`	Returns the caller's current credit balance and account identifier.	Yes	Any authenticated user
GET	`/api/credits/history?limit=\dots\&offset=\dots`	Returns the caller's credit ledger history using limit-offset pagination.	Yes	Any authenticated user
GET	`/api/credits/teachers`	Lists professor accounts with balances for the admin grant workflow.	Yes	Admin only
GET	`/api/credits/students`	Lists student accounts with balances for allocation workflows.	Yes	Admin or professor
POST	`/api/credits/allocate`	Grants or reallocates credits, with semantics that differ for admin and professor roles.	Yes	Admin or professor
GET	`/api/usage/\{pod\_id\}?range=1h\textbar6h\textbar24h\textbar7d`	Returns historical usage series for a specific pod.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/usage/summary/me/series?range=1h\textbar6h\textbar24h\textbar7d`	Returns aggregated historical usage across all of the caller's pods.	Yes	Any authenticated user
GET	`/api/usage/summary/me`	Returns a recent usage summary for the caller.	Yes	Any authenticated user

Administration endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/admin/users`	Lists all users with role and teacher-approval metadata.	Yes	Admin only
GET	`/api/admin/teacher-requests`	Lists pending teacher sign-up requests.	Yes	Admin only
POST	`/api/admin/teacher-requests/\{user\_id\}/approve`	Promotes a pending teacher request to the professor role.	Yes	Admin only
POST	`/api/admin/teacher-requests/\{user\_id\}/reject`	Rejects a pending teacher request while keeping the user as student.	Yes	Admin only
GET	`/api/admin/courses`	Returns the current course list. The implementation is presently an empty proof-of-concept response.	Yes	Admin only
GET	`/api/admin/nodes`	Returns cluster node capacity and readiness data via the orchestrator.	Yes	Admin only
GET	`/api/admin/stats`	Returns aggregate dashboard statistics such as total users and active VMs.	Yes	Admin only
GET	`/api/admin/active-vms`	Lists all currently active VM sessions together with owner details.	Yes	Admin only
PATCH	`/api/admin/users/\{user\_id\}/role`	Changes a user's role with safeguards against invalid or dangerous demotions.	Yes	Admin only
GET	`/api/admin/audit-logs?limit=\dots\&offset=\dots`	Returns recent audit-log entries using limit-offset pagination.	Yes	Admin only

Files, SSH keys, settings, notifications, and issue tracking

Method	Route	Purpose	Auth	Authorized Role
PUT	`/api/settings/vscode`	Accepts a VS Code settings blob. The current implementation acknowledges the request but still documents future persistence work in comments.	Yes	Any authenticated user
GET	`/api/ssh-keys/`	Lists the caller's SSH public keys.	Yes	Any authenticated user
POST	`/api/ssh-keys/`	Adds an SSH public key after format, duplicate, and per-user quota checks.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
DELETE	/api/ssh-keys/\{key_id\}	Deletes one of the caller's SSH keys.	Yes	Key owner
GET	/api/files/\{pod_id\}/list	Lists files inside a running pod directory.	Yes	Pod owner
POST	/api/files/\{pod_id\}/upload	Uploads a file to a running pod using multipart form submission and SCP behind the gateway.	Yes	Pod owner
GET	/api/files/\{pod_id\}/download	Downloads a file from a running pod as an octet-stream attachment.	Yes	Pod owner
POST	/api/issues/	Creates a user issue report, optionally linked to a pod.	Yes	Any authenticated user
GET	/api/issues/me	Lists issue reports submitted by the current user.	Yes	Any authenticated user
GET	/api/issues/admin?status=\dots	Lists issue reports for administrative triage, optionally filtered by status.	Yes	Admin only
POST	/api/issues/admin/\{issue_id\}/resolve	Marks an issue report as resolved.	Yes	Admin only
GET	/api/notifications/	Returns recent notifications together with an unread count.	Yes	Any authenticated user
POST	/api/notifications/read-all	Marks all of the caller's notifications as read.	Yes	Any authenticated user
POST	/api/notifications/\{notification_id\}/read	Marks one notification as read.	Yes	Notification owner
GET	/api/notifications/stream	Streams live notifications to the browser using Server-Sent Events.	Yes	Any authenticated user

3.3 Swagger / Postman Collection Link

The FastAPI gateway exposes the default Swagger UI, and the repository's ingress configuration makes it reachable publicly at https://hopper.farefin.com/api/docs. This URL is verified from the combination of the gateway's default FastAPI documentation settings and the ingress rule that forwards the external `/api` prefix to the backend service.

The public Postman Documenter page is available at https://documenter.getpostman.com/view/56891420/2sBY4QtLGS. Together, the Swagger UI and the published Postman collection provide both an implementation-native API explorer and a shareable external API reference.

3.4 Sample Request & Response

Example 1: direct login request

The following example reflects the request body accepted by `LoginRequest` in `services/api-gateway/app/schemas/user.py`. On success, the handler also sets secure HttpOnly session cookies; those headers are omitted here for brevity and because tokens

must not be reproduced in a report.

Listing 1: Representative login request to the Hopper API

```
curl -X POST "https://hopper.farefin.com/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "student@example.edu",
    "password": "[REDACTED]"
  }'
```

Listing 2: Representative login response body

```
{
  "id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "email": "student@example.edu",
  "name": "Example Student",
  "role": "student"
}
```

Example 2: pod creation request

This second example shows a provision request based on the verified `CreatePodRequest` and `PodResponse` schemas. The route may return `201 Created` for immediate admission or `202 Accepted` when the request is queued; the JSON below illustrates the immediate-creation path.

Listing 3: Representative pod creation request

```
curl -X POST "https://hopper.farefin.com/api/pods/" \
  -H "Content-Type: application/json" \
  -H "Cookie: session_token=[REDACTED]" \
  -d '{
    "plan": "small",
    "template": "ubuntu"
  }'
```

Listing 4: Representative pod creation response body

```
{
  "id": "9bc3d6ee-4b7a-4482-a9de-4948fdde2ce1",
  "user_id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "state": "running",
  "plan": "small",
  "image": "hopper/vm-ubuntu:22.04",
  "cpu": "1",
  "memory": "2Gi",
  "node_name": "worker-1",
```

```
  "namespace": "hopper-user-2f3ac4d8",
  "ssh_port": 32022,
  "vscode_port": 32080,
  "ssh_password": "[REDACTED]",
  "network_group": null,
  "created_at": "2026-07-19T12:00:00Z",
  "updated_at": "2026-07-19T12:00:30Z"
}
```

Example 3: queued pod creation response

When cluster capacity is temporarily exhausted, the same `POST/api/pods/` request is accepted into the admission queue instead of being created immediately. The API then returns `202 Accepted` with queue metadata so the frontend can show live position updates.

Listing 5: Representative queued pod creation request

```
curl -X POST "https://hopper.farefin.com/api/pods/" \
  -H "Content-Type: application/json" \
  -H "Cookie: session_token=[REDACTED]" \
  -d '{
    "plan": "medium",
    "template": "python"
  }'
```

Listing 6: Representative queued pod creation response body

```
{
  "id": "5d477c6d-8f51-4be6-8e3e-431c1bd0a0d2",
  "state": "queued",
  "plan": "medium",
  "position": 2,
  "queued": true
}
```

Example 4: credit balance request

The platform also exposes a simple authenticated read endpoint for wallet state. The following example reflects the `GET/api/credits/balance` route and the `CreditBalanceResponse` schema used by the student and teacher dashboards.

Listing 7: Representative credit balance request

```
curl -X GET "https://hopper.farefin.com/api/credits/balance" \
  -H "Accept: application/json" \
  -H "Cookie: session_token=[REDACTED]"
```

Listing 8: Representative credit balance response body

```
{
  "account_id": "7aa20ec1-5f52-4676-8a7e-8dd6de3b8fc1",
  "balance": 42.5
}
```

Example 5: queue status lookup

After a pod request has been queued, the frontend can poll `GET/api/pods/queue` to display current waiting jobs and their positions. This is the route used to keep the queue page synchronized with backend state.

Listing 9: Representative queue status request

```
curl -X GET "https://hopper.farefin.com/api/pods/queue" \
  -H "Accept: application/json" \
  -H "Cookie: session_token=[REDACTED]"
```

Listing 10: Representative queue status response body

```
[
  {
    "id": "5d477c6d-8f51-4be6-8e3e-431c1bd0a0d2",
    "plan": "medium",
    "template": "python",
    "state": "queued",
    "position": 2,
    "created_at": "2026-07-19T12:05:00Z"
  }
]
```

Example 6: cluster availability request

To help users understand whether a VM is likely to start immediately, Hopper exposes a summarized capacity endpoint at `GET/api/pods/availability`. The values below illustrate the normal case in which the orchestrator is reachable and aggregate resource metrics are available.

Listing 11: Representative availability request

```
curl -X GET "https://hopper.farefin.com/api/pods/availability" \
  -H "Accept: application/json" \
  -H "Cookie: session_token=[REDACTED]"
```

Listing 12: Representative availability response body

```
{
  "cpu": {
```

```
    "total_cores": 16.0,
    "used_cores": 10.0,
    "free_cores": 6.0
  },
  "memory": {
    "total_gib": 64.0,
    "used_gib": 40.0,
    "free_gib": 24.0
  },
  "storage": {
    "total_gib": 500.0,
    "used_gib": 190.0,
    "free_gib": 310.0
  },
  "nodes_ready": 3,
  "queue_length": 2
}
```


Example 7: notification feed request

The notification center is backed by `GET/api/notifications/`, which returns a recent rolling window of events together with the unread count shown in the UI badge.

Listing 13: Representative notifications request

```
curl -X GET "https://hopper.farefin.com/api/notifications/?limit=2" \
  -H "Accept: application/json" \
  -H "Cookie: session_token=[REDACTED]"
```

Listing 14: Representative notifications response body


```
{
  "notifications": [
    {
      "id": "c63a0b4f-130b-41af-b843-cb9b32f37d5e",
      "type": "success",
      "title": "Credits received",
      "body": "You received 10 credits from Prof. Rahman.",
      "data": {
        "amount": 10,
        "from": "Prof. Rahman"
      },
      "read": false,
      "created_at": "2026-07-19T12:08:00Z"
    },
    {
      "id": "91c7ce7d-5c9f-4141-998c-97f7a83ee0ab",
```

```
      "type": "info",
      "title": "VM ready",
      "body": "Your VM is now running and ready for SSH access.",
      "data": {
        "pod_id": "9bc3d6ee-4b7a-4482-a9de-4948fdde2ce1"
      },
      "read": true,
      "created_at": "2026-07-19T12:00:31Z"
    }
  ],
  "unread_count": 1
}
```


Example 8: SSH key upload request

Hopper allows users to register public keys for passwordless access through `POST/api/ssh-keys/`. The API stores the submitted key, derives a SHA-256 fingerprint, and returns the new identifier on success.

Listing 15: Representative SSH key upload request

```
curl -X POST "https://hopper.farefin.com/api/ssh-keys/" \
  -H "Content-Type: application/json" \
  -H "Cookie: session_token=[REDACTED]" \
  -d '{
    "name": "laptop-ed25519",
    "public_key": "ssh-ed25519
    AAAAC3NzaC1lZDI1NTE5AAAAIBExampleKeyMaterialForReportOnly user@laptop"
  }'
```

Listing 16: Representative SSH key upload response body


```
{
  "id": "4e2f4d25-9c2f-4e6f-8d45-215fe8ac2fcb",
  "fingerprint": "SHA256:
    Zk9kYVh6R3E0b2NQdXl0S3lBbjJxQ0h4R0l2Q0JmR2J5d1NxS3pPWT0"
}
```


4. Authentication & Security

This section documents security mechanisms verified in the Hopper repository as of the inspected tree. It does not assert that the platform is free of vulnerabilities; gaps called out below were confirmed by reading application code, Kubernetes manifests, CI workflows, and automated tests.

4.1 Authentication Strategy

Hopper delegates identity to **Keycloak** (OpenID Connect). The FastAPI gateway in `services/api-gateway` is the application entry point: it exchanges credentials with Keycloak, validates Keycloak-issued JWT access tokens, and exposes HTTP APIs to the SvelteKit frontend.

Dual sign-in paths. The repository implements two complementary browser flows:

- **OIDC redirect (SSO).** `GET/api/auth/login` generates PKCE (`S256`), `state`, and `nonce`, stores the PKCE verifier and state in short-lived HttpOnly cookies, and redirects the browser to Keycloak. `GET/api/auth/callback` verifies `state`, completes the authorization-code exchange with the PKCE verifier, validates the access token, upserts a local `users` row, and sets session cookies before redirecting to the dashboard.
- **Direct email/password login.** `POST/api/auth/login` uses Keycloak's resource-owner password grant against the `hopper-api` client (documented as requiring Direct Access Grants). On success it issues the same cookie set as the callback path. This supports the themed in-app login form without redirecting to Keycloak's hosted page.

Token and session lifecycle. Successful authentication sets HttpOnly, `Secure`, `SameSite=Lax` cookies:

- `session_token` — Keycloak access token (JWT); cookie `max-age` tracks Keycloak's `expires_in` (typically on the order of minutes).
- `refresh_token` — used by `POST/api/auth/refresh` to mint a new access token; refresh cookie lifetime tracks Keycloak's refresh TTL.
- `id_token` — retained for RP-initiated logout (`id_token_hint`).

One-time OAuth helper cookies (`oauth_state`, `oauth_pkce`) are cleared after callback. `POST/api/auth/refresh` reads the refresh cookie, calls Keycloak's token endpoint, rotates cookies on success, and clears all session cookies on failure. `POST/api/auth/logout` best-effort revokes the refresh token at Keycloak, clears local cookies, and redirects to Keycloak's end-session endpoint.

JWT validation. `app/middleware/auth.py` validates every access token with JWKS from Keycloak (`/protocol/openid-connect/certs`), checking signature, `exp`, issuer (external Keycloak URL), and audience (`hopper-api`). Realm roles are mapped to application roles `admin`, `professor`, or `student` (defaulting to `student` if no app role is present). JWKS is cached with periodic refresh and a forced refresh on unknown `kid` (key rotation).

Self-service account lifecycle. Signup (`POST/api/auth/signup`) creates users in Keycloak via the admin service account, stores a local row (teacher signups set `pending_teacher`), and emails a numeric verification code—*without* issuing a session until the

mailbox is verified when `HOPPER_REQUIRE_EMAIL_VERIFIED` is enabled (default `true`). Verification codes and password-reset codes are stored only as SHA-256 hashes with TTL and attempt limits (`app/services/verification.py`). Password rules are mirrored in `app/services/password_policy.py` before Keycloak enforces the realm policy.

Programmatic access. Scoped API keys (`hop_` prefix) authenticate via `X-API-Key` or `Authorization:Bearerhop_...`. Only a hash of each key is stored; plaintext is returned once at creation. Key management endpoints require a real cookie session, not an API key.

Frontend session handling. Server-side rendering in `frontend/src/routes/+layout.server.ts` loads the user via `/auth/me`, forwards `Set-Cookie` headers from `/auth/refresh` when the access token is stale, and exposes `isAuthenticated` to child routes. The browser client (`frontend/src/lib/api/client.ts`) sends credentialed requests, retries once after silent refresh on eligible `401` responses, and redirects to `/login?session_expired=1` when refresh fails. E2E tests (`tests/e2e/specs/auth-and-access.spec.ts`) assert HttpOnly session cookies and cross-tenant API denial.

Local development exception. `frontend/src/routes/dev-login/+server.ts` performs a password grant and sets cookies only when SvelteKit `dev` mode is active; production builds return `404`. Default usernames in that handler are overridable via private environment variables and are intended for local workflows, not production.

4.2 Authorization & Role Management

Role model. Three application roles are defined consistently in Keycloak realm configuration, JWT parsing, and the `users` table: `student`, `professor`, and `admin`. Keycloak remains canonical for role assignment; admin actions call `app/services/keycloak_admin.py` to promote users (for example, approving a pending teacher to `professor`).

Backend enforcement. Protected API routes depend on `get_current_user()` (`app/dependencies.py`), which resolves either a verified session JWT or a valid API key to a `TokenPayload`. Representative checks verified in routers include:

- **Admin-only:** `/api/admin/*` via `_require_admin` (professors are explicitly rejected).
- **Professor or admin:** credit listing/allocation endpoints in `app/routers/credits.py`; `network_group` on pod create is limited to professor/admin in `app/routers/pods.py`.
- **Resource ownership:** pod, file, terminal, and VS Code proxy routes compare `PodSession.user_id` to the authenticated subject; queue entries and SSH keys are scoped the same way.
- **API key scope:** `read_only` keys are rejected for non-safe HTTP methods; `full_access` keys inherit the owner's role but cannot manage API keys.

Frontend route guards. Authenticated pages redirect anonymous users to `/login`

from `+page.server.ts` loaders (dashboard, pods, credits, settings, and others). `admin/+page.server.ts` requires `user.role==='admin'`; `teacher/+page.server.ts` requires `professor`. These guards improve UX but are not a substitute for backend checks; the API enforces authorization independently (confirmed by E2E cross-tenant pod access tests returning `403` or `404`).

Teacher approval workflow. Users who sign up as "teacher" are stored as students with `pending_teacher=true` until an admin approves or rejects the request via `/api/admin/teacher-requests/*`. Role changes that would remove the last admin are blocked in the admin router.

4.3 Security Measures

Transport and edge hardening. Production ingress (`k8s/deploy/03-ingress.yaml`) enforces TLS redirect and sets HSTS, `X-Frame-Options`, `X-Content-Type-Options`, and `Referrer-Policy` on public paths. The API gateway adds `X-Content-Type-Options` and `Referrer-Policy` on all responses (`app/main.py`). The SvelteKit server hook (`frontend/src/hooks.server.ts`) adds `nosniff`, `DENY` framing, and referrer policy on SSR responses where ingress snippets are unavailable.

CORS. `CORSMiddleware` allows an explicit origin list from `HOPPER_CORS_ORIGINS` with credentials enabled. Startup fails if `*` appears in the list alongside credentialed cookies. Production manifests set a single origin (`https://hopper.farefin.com`). `/auth/refresh` additionally rejects requests whose `Origin` header is not in the allowlist.

Rate limiting and abuse controls. SlowAPI decorators cap sensitive unauthenticated routes (login, signup, verification, API-key creation). After authentication, `get_current_user` applies per-user (`user:<sub>`) or per-key (`apikey:<id>`) moving-window limits via in-memory storage (`app/core/limiter.py`). Failed password attempts are counted per `(client IP, email)` tuple before returning `429`. Pod creation has a separate per-user limit (`HOPPER_RATE_LIMIT_POD_CREATE`).

Input validation and sanitization. Request bodies use Pydantic models with length bounds (`app/schemas/user.py`, pod/credit schemas). Email domains can be restricted via `HOPPER_ALLOWED_EMAIL_DOMAINS` with exact suffix matching (not substring). File paths reject null bytes and newlines; remote paths are passed through `shlex.quote` before SSH (`app/routers/files.py`). SSH public keys are validated by prefix and fingerprint; per-user key count is capped with a transactional advisory lock.

Audit and error handling. `AuditMiddleware` logs mutating HTTP methods to `audit_logs` using user IDs derived from *verified* JWTs only. API errors generally return FastAPI JSON `{"detail":"..."}` with conventional status codes (`401`, `403`, `429`, etc.). Forgot-password and resend-code endpoints return uniform success responses to reduce email enumeration.

Secrets and configuration. Runtime settings load from environment variables prefixed

with `HOPPER_` (`app/config.py`, `services/api-gateway/.env.example`). Sensitive values intended for production—database URL, Keycloak admin client secret, Brevo API key, SMTP credentials—are documented as Kubernetes `Secret` references in `k8s/deploy/02-apps.yaml` rather than plain manifest env literals. GitHub Actions deploy and staging jobs consume repository secrets (`.github/workflows/publish.yml`, `staging-validation.yml`) for SSH, tokens, and staging URLs; workflow files reference secret *names* only, not values.

Repository inspection for hardcoded credentials. Application Python/TypeScript sources do not embed production API keys or live passwords. `app/config.py` and `k8s/deploy/00-secrets.yaml` contain *documented local-dev defaults* (for example `hopper_dev` database credentials and `REPLACE_ME` placeholders) that must be rotated before non-dev deployment; comments in `00-secrets.yaml` warn against applying dev secrets over production. E2E bootstrap scripts read test passwords from environment variables with test-only fallbacks (`scripts/test/bootstrap_keycloak.py`). Load-test scripts expect tokens via environment variables, not committed secrets.

File upload and interactive access. Uploads require authentication, pod ownership, and a running pod; content is streamed to a temporary file and copied via SCP over an SSH port-forward. The repository does *not* define a maximum upload size or content-type allowlist on the gateway—a confirmed gap (see Section 4.4). Terminal and VS Code proxies re-verify JWT cookies (and CORS origin on WebSockets) at the gateway before forwarding to user pods.

Network isolation (deployment layer). Multi-tenant `NetworkPolicy` and `CiliumNetworkPolicy` manifests under `k8s/deploy/04-network-policies.yaml` and `k8s/network-policies/` restrict pod-to-pod and egress traffic; infrastructure security tests in `tests/security/` document how to run authorization and isolation probes against a staging cluster.

4.4 Known Vulnerabilities & Mitigations

The table below separates protections already present in code or manifests from limitations observed during inspection and recommended follow-ups. None of these rows imply complete elimination of the listed risk class.

Risk	Existing Protection	Remaining Limitation	Recommended Mitigation
Stolen session or refresh token	Short-lived access cookies; HttpOnly+`Secure` flags; refresh origin check; RP-initiated logout with revoke	No server-side session revocation list beyond Keycloak refresh revoke; XSS in frontend could still exfiltrate cookies if `Secure` were disabled	Keep access TTLs short; monitor Keycloak sessions; consider tightening CSP; continue HttpOnly-only token storage (verified in E2E)
Credential stuffing / login brute force	Per-route SlowAPI caps; failed-attempt window per IP+email; generic `401` message	In-memory buckets scale with worker×replica count and reset on pod restart	Move rate-limit state to Redis or ingress-level throttling for consistent cluster-wide ceilings

Risk	Existing Protection	Remaining Limitation	Recommended Mitigation
Cross-user data access (pods, files, metrics)	`get_current_user` on routes; `user_id` ownership checks; E2E cross-tenant denial tests	Frontend role checks alone would be bypassable without API enforcement (API is enforced, but UI must stay aligned)	Keep adding integration tests for new resources; treat SSR guards as UX-only
API key leakage	Stored as SHA-256 only; read-only scope; separate rate limit; keys cannot create keys	`full_access` keys equal session power for automation; plaintext shown once at create	Prefer read-only keys where possible; rotate/revoke on suspicion; audit `api_key.*` actions
Weak or reused passwords	Keycloak realm policy + gateway mirror; history enforced in Keycloak on reset	Password grant and Direct Access Grants increase exposure if client misconfigured	Prefer authorization-code+PKCE for human login where feasible; disable password grant in production clients if SSO-only
Email / account enumeration	Forgot-password and resend-code always return `200`	Signup returns `409` on duplicate email; login returns distinct `403` for unverified email	Accept trade-off or unify error copy further; ensure production sets domain allowlists
CSRF on cookie-authenticated POST	`SameSite=Lax`; OIDC `state`; refresh `Origin` allowlist	No double-submit CSRF token on all mutating forms; `Lax` cookies still sent on top-level cross-site GET navigations	Add CSRF tokens for sensitive POSTs if third-party integrations expand
Large or malicious file upload	AuthZ on pod owner; path sanitization	No max upload size; full file buffered to temp storage; filename not sanitized for path traversal on remote join	Enforce size limits at gateway/ingress; validate filenames; stream with backpressure
MITM on gateway→VM SSH	Access limited to authenticated owners	`StrictHostKeyChecking=no` for automated SCP/SSH from gateway	Use known-hosts pinning or in-cluster exec without password-based SSH where possible
Default or committed secrets	K8s Secret refs for prod DB/SMTP/Brevo/Keycloak admin; `.env.example` documents vars without live values	`00-secrets.yaml` ships dev defaults in git; `HOPPER_CODE_URL_SIGNING_SECRET` default in code; setting is unused in routers inspected; `HOPPER_DEBUG=true` in `02-apps.yaml`	Provision secrets out-of-band for prod; wire signing secret via Secret if feature is enabled; set `HOPPER_DEBUG=false` in production
Public API surface disclosure	Auth on sensitive routes	Swagger UI at `/api/docs` is publicly reachable per ingress (Section 3)	Restrict docs to authenticated admins or internal networks if required by policy
Multi-tenant network escape	Cilium/NetworkPolicy manifests; optional `tests/security/` suite	Policies require correct cluster CNI deployment; not re-verified in this report compile	Run `tests/security/run-security.sh` on staging; keep network policies in sync with orchestrator labels

Automated security test coverage. Unit tests cover password-policy errors, API-key dependency behavior, rate-limiter keying, and pod authorization edge cases (`tests/unit/`). E2E specs cover login cookies, role-scoped navigation, silent refresh, and anonymous `401` responses. Broader infrastructure isolation tests are documented but require a dedicated staging cluster and tokens supplied via environment variables (`tests/security/README.md`).

5. Testing & Quality Assurance

5.1 Testing Strategy

Hopper adopts a layered testing pyramid that keeps the majority of checks fast and GPU-independent. The strategy is captured in `docs/TESTING_PLAN.md` and operationalised through the CI pipeline in `.github/workflows/ci.yml`.

Unit tests (Python/pytest) target the API Gateway in isolation. All 411 cases run without any live infrastructure; every service, router, schema, and middleware module has dedicated coverage files under `tests/unit/` (47 Python modules). Key focus areas include the double-entry credit ledger, billing consumer state machine, password-policy enforcement, RBAC dependency checks, and session management.

Unit tests (Go/testing) exercise the orchestrator's internal packages and billing-contract layer using fake Kubernetes client-go sets and in-process stubs. Sixteen tests pass in `go-test.json` (internal K8s package), with a further 27 contract-level passes recorded in `go-contracts.json` (billing pricing, pod lifecycle state transitions, and resource-limit enforcement).

Integration tests (Python/pytest) in `tests/integration/` cover 18 modules against live Docker-based PostgreSQL, NATS JetStream, and Keycloak services spun up by the `python-integration` CI job. Modules include `test_credit_ledger_invariants`, `test_nats_event_flow`, `test_keycloak_integration`, `test_pods_api`, `test_auth_api`, and 13 others.

Frontend unit tests (Vitest) in `frontend/src/` verify 78 test cases across route server loads, API client behaviour, and Svelte component logic (including auth redirect rules, session refresh paths, and landing-page server guards).

End-to-end tests (Playwright) in `tests/e2e/specs/` span 6 specification files and 35 test cases as documented in `tests/e2e/COVERAGE_MATRIX.md`. Tests run against a mock-backed stack (SvelteKit dev server + Python mock API server) under the `e2e-mock` CI job. The final mock-suite run reported **27 passed (25.9s)**.

Load tests (k6) in `tests/load/` define five scenarios: class-start concurrent pod creation (30 VUs), steady-state metrics polling (100 VUs), spike to 150 VUs, class-end mass termination, and billing stress. Results are stored in `test-results/load/`. The scenarios run recorded 81 456 passing spike-check iterations; pod creation completed under 2 s on all sampled requests.

Helm chart validation runs on every push: `helm lint` and `helm template` are executed against `dev`, `staging`, and `prod` value overlays. Infrastructure chaos and security manifests reside under `tests/chaos/` and `tests/security/` for manual staging runs.

5.2 Test Coverage Summary

Coverage is measured independently per layer and aggregated by the `coverage-report` CI job. Figures are sourced from the artifacts recorded on **2026-07-18**.

Table 10: Automated test results and coverage by layer

Layer	Tests	Coverage	Artifact
API Gateway — Python unit	411	81.68 %	`coverage/python/unit.xml`
API Gateway — Python integration	18 modules	—	`tests/integration/`
Frontend — SvelteKit with Vitest	78	66.08 %	`coverage/frontend/coverage-summary.json`
Orchestrator — Go internal tests	16	71.26 %	`coverage/orchestrator/orchestrator.out`
Orchestrator — Go contract tests	27	—	`test-results/contracts/go-contracts.json`
End-to-end — Playwright mock suite	27 passed	—	`test-results/e2e/playwright-junit.xml`
Load testing — k6 spike scenario	81 456 checks	—	`test-results/load/scenarios-summary.json`

The lowest-covered API Gateway files are `routers/terminal.py` (41.90 %) and `routers/pods.py` (58.80 %). On the frontend, four files have 0 % coverage: `stores/notifications.ts`, `routes/dev-login/+server.ts`, `lib/confirm.svelte.ts`, and `routes/pods/queue/+page.server.ts`. In the orchestrator, `internal/k8s/pod.go` is the lowest at 57.80 %; the network-policy module (`netpol.go`) is at 100 %.

Confirmed testing gaps. The following areas currently lack automated coverage:

- Integration coverage XML (`coverage/python/integration.xml`) was not generated in the recorded run; Python integration tests exercise live services but no statement-level coverage artifact was produced.
- E2E test cases `TC-CREDIT-003` (grace-period auto-termination), `TC-ADMIN-003` (course creation UI), `TC-EDGE-003` (session-duration expiry), `TC-TMPL-002` (course template management), `TC-SCAV-001`, and `TC-SCAV-002` (scavenger plans) are marked *pending* or *future* in `tests/e2e/COVERAGE_MATRIX.md`.
- Cross-browser E2E is not enforced by default; Firefox and WebKit configurations exist but are opt-in via `E2E_CROSS_BROWSER`.
- Chaos experiments in `tests/chaos/` and security manifests in `tests/security/` require a live staging cluster and are excluded from the automated CI pipeline.

5.3 Bug Tracking & Resolution Log

All defects discovered during testing were tracked in `BUG_TRACKING_LOG.md`. The log covers 2026-07-17 to 2026-07-18 and records **20 bugs**, all with status *Resolved*. Table 11 lists every entry.

Table 11: All bugs tracked in `BUG_TRACKING_LOG.md` (20 entries, all Resolved)

Bug ID	**Component**	**Root cause**	**Fix applied**
BUG-2026-07-17-01	Frontend tabs	`Tabs.Trigger` did not forward extra props, so per-page `onclick` handlers were swallowed before reaching `Bits.Trigger`.	Forwarded `...restProps` in `tabs-trigger.svelte`; added explicit `onclick` state updates on admin and pod-detail tab triggers.
BUG-2026-07-17-02	Frontend terminal toolbar	"New terminal" control was wrapped in a tooltip snippet, lacking stable button semantics for E2E.	Replaced with a plain `<button>` carrying `type="button"`, `aria-label`, and `title`.
BUG-2026-07-17-03	E2E admin spec	`getByText(...).first()` resolved hidden `md:hidden` fragments, producing false negatives on visible rows.	Added hydration waits; replaced text-only assertions with `getByRole('row').filter(...)`.
BUG-2026-07-17-04	E2E pod lifecycle spec	Tests assumed stricter conditions than the mock harness guaranteed (live metrics, history-card identity).	Adjusted specs to wait for stable rendered states and verify termination at the API boundary.
BUG-2026-07-18-01	Frontend auth guards	All unauthenticated SSR loads redirected to `/login?session_expired=1`, falsely implying an expired session.	Protected routes now redirect to plain `/login`; `session_expired` flag reserved for real client-side refresh failures.
BUG-2026-07-18-02	CI workflow `ci.yml`	`upload-artifact` pointed at fixed paths with `if-no-files-found: error`, breaking the job when coverage output varied.	Added a staging step that copies only existing artifacts; uploads with `if-no-files-found: warn`.
BUG-2026-07-18-03	`scripts/test.sh`	`go tool cover -html` ran from the repo root, breaking module-relative resolution for orchestrator reports.	Coverage renderer `cd`s into `services/orchestrator` before invoking `go tool cover`.
BUG-2026-07-18-04	Frontend build pipeline	`build` script called `vite build` directly without syncing SvelteKit generated files first.	Changed build script to run `svelte-kit sync && vite build`.

Table 11 continued

Bug ID	Component	Root cause	Fix applied
BUG-2026-07-18-05	E2E auth smoke	`TC-AUTH-001` asserted stale copy (`Welcome back`, `Sign in as admin`) that no longer matched the login page.	Updated spec to assert the current login page contract (`Sign in with email`, `University SSO`).
BUG-2026-07-18-06	E2E session refresh	`TC-AUTH-004` expected `?session_expired=1` redirect that the app no longer produces for plain unauthenticated access.	Updated auth spec to expect plain `/login` for the expired-refresh fallback.
BUG-2026-07-18-07	E2E pod launch	`TC-POD-001` used the hidden terminal control as a readiness signal; page now opens on overview tab.	Rewrote pod-detail readiness to wait for pod heading and selected overview tab.
BUG-2026-07-18-08	E2E live metrics	`TC-POD-003` used the wrong readiness signal and stale text casing for the metrics panel.	Updated spec to use current overview readiness contract and `Live Metrics` copy.
BUG-2026-07-18-09	E2E pod termination	`TC-POD-004` never reached termination assertions because the shared helper waited on the wrong terminal control.	Reused the corrected pod-detail readiness helper across all pod-lifecycle tests.
BUG-2026-07-18-10	E2E queue routing	`TC-EDGE-005` asserted a `Position` column that no longer exists; cancel selector matched multiple buttons.	Updated to assert the current queue page contract; scoped cancel control to `main`.
BUG-2026-07-18-11	E2E performance smoke	Spec used old browser-form login path and required `/pods` navigation under 500 ms (actual: ~863 ms).	Switched to direct API login; relaxed pods-navigation threshold to <1000 ms.
BUG-2026-07-18-12	E2E terminal clipboard	`TC-TERM-004` targeted xterm's hidden helper `<textarea>` instead of the visible terminal surface.	Added `?tab=terminal` deep-link support; clipboard spec opens the terminal tab directly.
BUG-2026-07-18-13	E2E terminal connect	Terminal tests raced hydration on the overview tab; `TC-TERM-001` intermittently never reached a visible xterm surface.	Added pod-page hydration marker and `?tab=terminal` direct entry; runtime tests open on terminal tab.

Table 11 continued

Bug ID	Component	Root cause	Fix applied
BUG-2026-07-18-14	E2E runtime workspace	After deep-linking to `?tab=terminal`, test still asserted `Live Metrics` without switching back to overview.	Updated spec to verify terminal controls first, then switch to Overview before asserting metrics.
BUG-2026-07-18-15	E2E terminal resize	`TC-TERM-002`/003 blocked by the same terminal-entry race as `TC-TERM-001`.	Reused corrected terminal-entry path (`?tab=terminal` + hydration wait) in the runtime spec.
BUG-2026-07-18-16	Frontend pods page	`TC-EDGE-001` was flaky because the launch button was interactable before Svelte attached the confirmation-dialog handler.	Added `/pods` hydration marker; E2E helpers wait for hydration before clicking *Launch VM*.
BUG-2026-07-18-17	Frontend root route	`+page.server.ts` redirected anonymous users from `/` to `/login`, hiding the landing page.	Removed anonymous redirect from root server load; tightened E2E landing-page assertion to real `<h1>` text.
BUG-2026-07-18-18	Landing page navbar	Every navbar link hard-coded to `#features`, so all items navigated to the same section.	Each link now carries a distinct `href` anchor; matching section IDs added to `+page.svelte`.
BUG-2026-07-18-19	Landing page header	`ThemeToggle` was only mounted inside the authenticated layout, leaving anonymous visitors without theme control.	Reused the existing `ThemeToggle` component in the landing-page header.
BUG-2026-07-18-20	Dashboard navigation	No loading skeleton during route transitions caused visible blank states between data-heavy pages.	Added shared `DashboardSkeleton` component; route-specific skeleton variants wired in `+layout.svelte`.

5.4 Sample Test Cases

The sample cases below are drawn directly from the repository test suites and are intentionally spread across unit, integration, end-to-end, security, and chaos coverage so that the report reflects the actual verification breadth of the codebase rather than only a single testing layer.

TC-1 — Billing consumer: insufficient-credit grace logic
Suite: `tests/unit/services/test_billing_consumer.py` — *passed* (pytest-junit 2026-07-18). Verifies that when a `billing.deducted` NATS message arrives and the credit deduction raises `InsufficientCredits`, the consumer invokes the grace-period logic rather than nakking or crashing. Test: `test_handle_billing_deducted_runs_grace_logic_on_insufficient_credits`.

TC-2 — Credit service: double-entry balance invariant
Suite: `tests/unit/services/test_credit_service.py` — *passed.* Confirms that `add_credits` produces a transfer record plus exactly two balanced ledger entries that sum to zero, preserving double-entry bookkeeping. Test: `test_add_credits_creates_transfer_and_balanced_entries`.

TC-3 — Orchestrator: pod lifecycle state transitions
Suite: `tests/orchestrator/lifecycle_test.go` — *passed in Go tests.* The Go suite exercises multiple transition rules. Two core sub-tests are: `TestPodLifecycleValidTransitions` asserts the legal path REQUESTED → CREATING → RUNNING → STOPPING → TERMINATED, and `TestPodLifecycleRejectsInvalidTransition` confirms that skipping required intermediate states is rejected. The same suite also checks failure transitions from active states and idempotent concurrent creation.

TC-4 — Orchestrator: VM plan billing rates
Suite: `tests/orchestrator/billing_pricing_test.go` — *passed in Go tests.* `TestVMPlanHourlyRates` checks Small=1, Medium=2, Large=4 credits/hour. `TestPartialHourBillingUsesPerMinuteRate` verifies per-minute pro-ration at session end, while `TestStopAndProrateProducesFinalIdempotencyKey` verifies that final billing events produce a stable deduplication key.

TC-5 — E2E: professor credit allocation (TC-CREDIT-004)
Suite: `tests/e2e/specs/credits-and-teaching.spec.ts` — *covered*; included in the final mock-suite run (27 passed, 25.9 s). A Playwright test logs in as a professor, navigates to the credits page, and allocates credits to a student account, then asserts the student's displayed balance increases.

TC-6 — E2E: capacity exhaustion routes to queue (TC-EDGE-005)
Suite: `tests/e2e/specs/pod-lifecycle.spec.ts` — *covered*; final mock-suite run passed. When all capacity is occupied, a launch request returns a queued response and the frontend redirects the user to `/pods/queue`. The test asserts the queue page renders the in-queue entry.

TC-7 — E2E: login sets secure session cookies (TC-AUTH-001)
Suite: `tests/e2e/specs/auth-and-access.spec.ts` — *covered.* This Playwright case signs in through the real application login form, confirms the user lands on the dashboard, and verifies that both `session_token` and `refresh_token` are issued as `HttpOnly` cookies.

TC-8 — E2E: cross-tenant isolation and unauthenticated API denial (TC-AUTH-003 / TC-AUTH-005)
Suite: `tests/e2e/specs/auth-and-access.spec.ts` — *covered.* The suite injects another student's pod into mock state, confirms a logged-in student receives `403` when requesting that pod through the API, and separately asserts that anonymous access to `/api/pods/` returns `401`.

TC-9 — E2E: terminal reconnect after WebSocket drop (TC-TERM-003)
Suite: `tests/e2e/specs/runtime-and-terminal.spec.ts` — *covered.* The mock terminal is configured to disconnect shortly after connect; the test then verifies the terminal surfaces a reconnecting state and later returns to a connected, usable shell.

TC-10 — Integration: pod creation persists runtime connection details
Suite: `tests/integration/test_pods_api.py` — *passed when Docker-backed integration tests are enabled.* `test_create_pod_persists_running_session_details` stubs the orchestrator response and verifies that a successful `POST /pods/` persists and returns the running state together with SSH port, VS Code port, and generated SSH password.

TC-11 — Integration: pod termination updates session state and tears down access paths
Suite: `tests/integration/test_pods_api.py` — *passed when Docker-backed integration tests are enabled.* `test_terminate_pod_marks_session_terminated` seeds a running session, invokes `DELETE /pods/id`, and asserts that the orchestrator termination call and local port-forward shutdown are both triggered.

TC-12 — Security: network and admission-policy isolation probes
Suite: `tests/security/run-security.sh` with `tests/security/test-network-isolation.sh` and manifest probes — *staging-oriented security coverage.* The security harness performs admission dry-runs for forbidden pod patterns such as `hostNetwork`, `hostPath`, and privileged containers, and also probes east-west isolation between student namespaces and internal services such as NATS and PostgreSQL.

TC-13 — Chaos/invariant verification: ledger conservation and running-session consistency
Suite: `tests/chaos/verify-invariants.sh` — *post-chaos verification coverage.* After chaos experiments, the script verifies three critical invariants: total signed ledger sum remains zero, no account has a negative latest balance, and the count of application sessions marked `running` matches the count of Kubernetes student pods currently running.

6. CI/CD & Deployment

6.1 Pipeline Overview

Hopper uses **GitHub Actions** for all continuous integration and delivery. Five workflow files live in `.github/workflows/`.

`ci.yml` — Continuous Integration

Triggered on every push or pull-request targeting `main`/`master`. A `concurrency` group cancels in-progress runs on the same ref. The workflow contains **twelve parallel-capable jobs**:

- **frontend** (20 min timeout) — installs pnpm 10 / Node 22, runs `frontend-validate` (TypeScript check + ESLint) and `test-frontend` (Vitest with V8 coverage), stages and uploads coverage artifacts with `if-no-files-found: warn`.
- **api-gateway-smoke** (15 min) — installs Poetry 1.8.4 / Python 3.12 and performs an import smoke test (`from app.main import app`).
- **python-unit** (20 min) — runs the full pytest unit suite (`test-unit`); uploads `coverage/python/unit.xml` and JUnit results.
- **python-migrations** (20 min) — spins up deterministic Docker services (`test-services-up`), runs Alembic migrations (`test-migrate`), captures logs, tears down.
- **python-integration** (30 min) — runs the full pytest integration suite against live containers; uploads XML artifacts.
- **go-orchestrator** (20 min) — runs orchestrator Go tests (`test-orchestrator`), Go 1.23; uploads coverprofile and JSON results.
- **contract-tests** (20 min) — runs both Python and Go contract suites (`test-contract`).
- **helm-chart** (10 min) — runs `helm lint` and `helm template` for `dev`, `staging`, and `prod` value overlays; uses Helm v3.19.0.
- **lint-go** (20 min) — generates proto stubs with `protoc`, then runs `golangci-lint v2.6.2` over both the orchestrator and the contract-test module.
- **e2e-mock** (30 min) — installs Playwright Chromium, starts the deterministic mock stack, runs the full mock-backed E2E suite, captures logs, uploads the Playwright report (retained 14 days).
- **coverage-report** — depends on `frontend`, `python-unit`, `python-integration`, `go-orchestrator`, and `contract-tests`; downloads all coverage artifacts and renders the combined Markdown report.
- **docker** (30 min) — builds all three service images via `scripts/ci/docker-build.sh` as a smoke check (no push).

`publish.yml` — Image Publish & Rollout (CD)

Triggered on push to `main`/`master`, version tags (`v*`), or manual `workflow_dispatch`. Two jobs:

- **build-push** — authenticates to **GHCR** (`ghcr.io`) with `GITHUB_TOKEN`, builds and pushes the three service images (`hopper-api-gateway`, `hopper-orchestrator`, `hopper-frontend`) tagged with the 12-character commit SHA (`ghcr.io/crevios/<image>:<sha12>`).
- **deploy** — conditional on `build-push` succeeding and either a manual `deploy=true` input or the repo variable `AUTO_DEPLOY_K8S=true`. SSHes into the VPS using `VPS_SSH_KEY`/`VPS_HOST`/`VPS_USER` secrets, pipes `scripts/cd/k8s-rollout.sh` over the SSH connection. The rollout script calls `kubectl set image` on `api-gateway`, `orchestrator`, and `frontend` deployments and waits for each rollout with a configurable timeout (default 900 s).

`test-platform.yml` — Extended Platform Tests

Triggered on push to `main`, nightly schedule (`17 2 * * *`), or manual dispatch. Four jobs: `integration-auth-events` (Keycloak + NATS integration suites), `real-stack-e2e` (full real-stack Playwright run with SvelteKit dev server, FastAPI, and Go orchestrator), `load-smoke` (k6 load smoke against the mock stack), and `cross-browser-real-stack-e2e` (opt-in; Chromium + Firefox + WebKit, enabled via `E2E_CROSS_BROWSER=true` or `CROSS_BROWSER_E2E` repo variable).

`staging-validation.yml` — Staging QA

Triggered on manual dispatch or weekly Saturday schedule (`30 3 * * 6`). Runs three parallel jobs against a live staging cluster: `staging-load` (k6 stress/spike/soak profiles against `STAGING_BASE_URL`), `security` (Python security checks with staging KUBECONFIG), and `chaos` (Chaos Mesh YAML manifests from `tests/chaos/`).

`lint-python.yml` runs Ruff on the API Gateway on every push.

ArgoCD (GitOps — partial). An app-of-apps ArgoCD configuration exists at `k8s/argocd/`. The root Application (`hopper-root`) points at `k8s/argocd/apps/`; the child `hopper-platform` deploys `charts/hopper` with `values-prod.yaml`. Sync is **deliberately manual** (no `automated` sync policy, no `prune`) to avoid self-heal reverting CI image rollouts and prune deleting out-of-band Secrets and dynamically-created VM pods. The current live CD path remains `publish.yml`'s SSH-based `kubectl set image`; ArgoCD provides drift visibility and one-click manual sync.

6.2 Environments

Three environments are defined via Helm value overlays in `charts/hopper/` and are verified by the `helm-chart` CI job on every push.

Local infrastructure (PostgreSQL/TimescaleDB, NATS JetStream, Keycloak) is managed by `docker-compose.yml`. Image digests are pinned (`timescale/timescaledb:latest-pg16@sha256:0af03...`) to prevent silent major upgrades.

Table 12: Deployment environments

Environment	Host and values file	Key differences
Development	`localhost` `values-dev.yaml`	Uses Keycloak `start-dev` and in-cluster development secrets. Ingress, TLS, network policies, and backups are disabled. Application services normally run through Docker Compose and `scripts/deploy-local.sh`.
Staging	`staging.hopper.farefin.com` `values-staging.yaml`	Uses one API Gateway replica and the Let's Encrypt staging issuer. Certificates are untrusted for production use, nightly backups are disabled, and secrets are provisioned separately.
Production	`hopper.farefin.com` `values-prod.yaml`	Uses two API Gateway replicas, the Let's Encrypt production issuer, Cilium network policies, nightly `pg_dump` backups with 14-day retention and a 10 Gi PVC, and an SMTP relay on port 2525.

6.3 Deployment Steps

The following procedure is extracted from `README.md`, `scripts/cd/k8s-rollout.sh`, and `k8s/deploy/`.

1. **Build images.** From the repository root:

```
TAG=$(git rev-parse --short HEAD)
REGISTRY=ghcr.io/crevios ./scripts/ci/docker-build.sh
```

   Images produced: `hopper-api-gateway:$TAG`, `hopper-orchestrator:$TAG`, and `hopper-frontend:$TAG`. The CI `publish.yml` workflow sets `DOCKER_PUSH=1` to push to GHCR automatically.

2. **Import VM template images** into the k3s containerd runtime (required once per node, not on every deploy):

```
docker save hopper/vm-ubuntu:22.04 | sudo k3s ctr images import -
```

3. **Apply Kubernetes manifests.** Initial cluster setup uses raw manifests in order:

```
kubectl create namespace hopper
kubectl apply -f k8s/deploy/01-infra.yaml    # PostgreSQL, NATS, Keycloak
kubectl apply -f k8s/deploy/02-apps.yaml     # API Gateway, Orchestrator,
    Frontend
kubectl apply -f k8s/deploy/03-ingress.yaml  # Nginx Ingress rules
kubectl apply -f k8s/deploy/04-network-policies.yaml  # Cilium zero-trust
kubectl apply -f k8s/deploy/06-backup.yaml   # nightly pg_dump CronJob
```

Subsequent deploys use `kubectl set image` via `scripts/cd/k8s-rollout.sh` (invoked by `publish.yml`).

4. **Run database migrations.**

```
docker run --rm --network host \
  -e HOPPER_DATABASE_URL="postgresql+asyncpg://hopper:..." \
  -e PYTHONPATH=/app \
  hopper/api-gateway:latest alembic upgrade head
```

Three Alembic migrations exist: initial schema (001), pod-session column fixes (002), plan/CPU/memory/SSH-port columns (003).

5. **Bootstrap Keycloak.** Create the `hopper` realm, disable SSL for POC deployments, add roles (`admin`, `professor`, `student`), create the initial admin user, and configure redirect URIs for the `hopper-api` public client. Commands use `kcadm.sh` executed inside the Keycloak pod via `kubectl exec`.

6. **Helm-based upgrade** (post-adoption):

```
helm upgrade --install hopper charts/hopper \
  -n hopper -f charts/hopper/values-prod.yaml
```

The chart manages infra, app deployments, ingress, NetworkPolicies, cert-manager ClusterIssuers, and the backup CronJob. Secrets are provisioned out-of-band and must exist before install (`secrets.create=false`).

Open Azure NSG ports (if hosted on Azure): port 443 (HTTPS), port 80 (HTTP), and the NodePort range 30000–32767 (SSH into user VMs) must be open in the Network Security Group.

6.4 Hosting Platform & Live URL

The production cluster is a **k3s** single-node Kubernetes cluster hosted on a **Microsoft Azure** virtual machine. The platform is accessible at `https://hopper.farefin.com`. TLS certificates are issued automatically by **cert-manager** using the Let's Encrypt production issuer (`letsencrypt-prod`), with the TLS secret `hopper-farefin-tls`. The Nginx Ingress Controller routes traffic to the frontend (port 3000), API Gateway (port 8000), and Keycloak (port 8080). Service images are stored in the GitHub Container Registry at `ghcr.io/crevios/` and tagged with the 12-character commit SHA. The current bootstrap image tag recorded in `values-prod.yaml` is `b19bbd699a03`.

6.5 Monitoring & Logging

Observability configuration lives under `observability/`.

Prometheus (`observability/prometheus/prometheus.yml`) scrapes the following targets every 15 s:

- `api-gateway:8000/metrics` and `orchestrator:50051/metrics` (application metrics).
- `nats:8222/metrics` (NATS JetStream consumer lag and message rates).
- `dcgm-exporter` (discovered via Kubernetes endpoint SD) for GPU temperature, VRAM utilisation, and GPU utilisation.
- `node-exporter` (discovered via Kubernetes node SD) for host-level CPU, memory, and disk metrics.

Prometheus is configured with `remote_write` to a **TimescaleDB** instance (the same PostgreSQL pod, via port 9201) for long-term metrics retention.

Alerting. Alert rules in `observability/prometheus/alerts/gpu-alerts.yml` fire to `alertmanager:9093`. Defined rules:

- **GPUTemperatureCritical** — `DCGM_FI_DEV_GPU_TEMP > 90` for 5 min (severity: critical).
- **GPUMemoryNearFull** — VRAM usage above 95% for 2 min (severity: warning).
- **GPUUtilizationStuck** — GPU at 100% for 30 min (severity: warning).
- **DCGMExporterDown** — exporter unreachable for 5 min (severity: critical).
- **PodStuckPending** — pod in `hopper-*` namespace pending for 10 min (severity: warning).
- **NodeUnreachable** — node not Ready for 5 min (severity: critical).
- **NATSJetStreamLag** — consumer pending messages $> 1\,000$ for 5 min (severity: warning).

Grafana dashboard configuration exists at `observability/grafana/dashboards/gpu-overview.json` (GPU overview). Grafana is not managed by the Helm chart and is deployed separately.

Loki (`observability/loki/loki-config.yml`) is configured with filesystem storage (TSDB schema v13), a 30-day log retention policy, and an in-memory ring KV store. Log ingestion rate is limited to 10 MB/s (burst 20 MB/s).

Structured logging is used throughout the API Gateway via `structlog`. The audit middleware records every mutating HTTP request (method, path, user ID, resource ID) as an immutable audit entry in PostgreSQL.

Health and readiness probes. The API Gateway exposes `/healthz` and `/readyz`. The `/readyz` endpoint checks PostgreSQL connectivity, NATS connectivity, and orchestrator reachability, reporting each dependency individually; an orchestrator outage is

reported but does not gate readiness (verified by `test_readyz_orchestrator_outage_is_reported_but_not_gating` in the unit suite).

Database backups. A Kubernetes CronJob (`pg-backup`, manifest `k8s/deploy/06-backup.yaml`) runs `pg_dump` nightly at 21:00 UTC using custom format (`-Fc`). Dumps are written to a 10 Gi `local-path` PVC and retained for 14 days.

7. Repository & Documentation Quality

7.1 Branching Strategy & Commit Conventions

Repository. The repository is hosted at `https://github.com/CREVIOS/Hopper`. The default integration branch is **`main`**. At the time of inspection there are 3 local branches (`main`, `Test`, `report`) and 20 remote branches visible via `git branch -a`.

Branch naming. Feature branches follow a `feat/<scope>` pattern (e.g. `feat/HOP-6-web-terminal`, `feat/cpu-vm-admission-queue`, `feat/external-ssh-fix-and-platform-updates`). Fix branches use `fix/<scope>` or plain descriptive names (e.g. `ektu-fix`, `cicd-again`). Documentation branches use `docs/<scope>` (e.g. `docs/vm-queueing`). A `Test` branch is used for iterative CI validation passes.

Merge model. All changes enter `main` via **pull requests**: the log shows 88 commits, the majority carrying a PR number suffix (`#66`–`#115`). GitHub Actions CI gates merges (push/PR trigger on `main`).

Commit conventions. The project uses a subset of **Conventional Commits**. Verified prefixes in the history:

- **`feat`** / **`feat(<scope>)`** — new capability, e.g. *feat(auth): open signup to any email + email verification & password reset (#71).*
- **`fix`** / **`fix(<scope>)`** — bug fix, e.g. *fix(billing): stop ResourceClosedError on idempotent credit deductions (#69).*
- **`docs`** / **`docs(<scope>)`** — documentation only, e.g. *docs(k8s): capture the SMTP relay workaround in the repo (#74).*
- **`chore`** / **`chore(<scope>)`** — maintenance, e.g. *chore: stop tracking scratchpad/ (#73).*
- **`revert`** — undo a previous commit.

Not every commit is prefixed; iterative CI passes use a plain `Test (#N)` message. No `CONTRIBUTING.md` or PR template file was found in `.github/` at the time of inspection.

Code style enforcement. An `.editorconfig` at the repository root enforces: 2-space indent for most files; 4-space for Python; tab indent for Go and Makefiles; LF line endings; UTF-8; trailing-whitespace trimming; final newline. Ruff lints the API Gateway on every

push via `lint-python.yml`. ESLint lints the frontend (run inside the `frontend` CI job). `golangci-lint v2.6.2` lints the orchestrator and contract-test modules (`lint-go` CI job). The `.gitignore` excludes `.env`/`.env.*` (with `!.env.example` kept), generated proto stubs, build artefacts, and local IDE directories.

7.2 README Completeness Checklist

The root `README.md` (11 704 bytes) covers the following topics, verified by direct inspection:

Table 13: README completeness audit

Topic	Present
Project description and purpose	✓
Architecture diagram (ASCII art)	✓
Services table (language, port, purpose)	✓
Key flows (VM creation, billing, metrics)	✓
VM plans table (CPU, memory, disk, credits/hour)	✓
Prerequisites list (Docker, kubectl, Go, Node, pnpm)	✓
Quick-start steps (infra, images, migrations, k8s deploy)	✓
Keycloak setup (realm, roles, test user, redirect URIs)	✓
Credit allocation example (curl commands)	✓
Azure NSG port table	✓
Network policies documentation (Cilium, 8 rules)	✓
NATS subject table (publisher, consumer, purpose)	✓
Keycloak OIDC configuration summary	✓
API endpoint reference (auth, pods, credits, admin)	✓
Proto contract paths	✓
Database migration list (3 Alembic versions)	✓
Double-entry ledger design note	✓
SSH-into-VM instructions	✓
Default credentials table (POC)	✓
Environment `.env` variable reference	Partial (`.env.example` only)
Testing instructions	Not present in README
Contributing / PR workflow guide	Not present

A per-service `README.md` exists for the frontend (`frontend/README.md`, 9 571 bytes) covering local dev setup, Vite proxy configuration, dev-login flow, WebSocket origin requirements, and terminal/VS Code connection notes. The `docs/` directory contains `ARCHITECTURE.md`, `SDD.md`, `TESTING_PLAN.md`, `E2E_TESTING.md`, `VM_QUEUEING.md`, `TECH_STACK.md`, `SRS_ADDENDUM.md`, and `TASKS.md`.

7.3 Code Organization / Folder Structure

The repository is organised around service and concern boundaries. The top-level layout (verified by directory listing) is:

Table 14: Top-level repository structure

Path	Contents / Purpose
services/api-gateway/	FastAPI application (`app/`), Alembic migrations, Poetry config, Dockerfile, `.env.example`
services/orchestrator/	Go module (`cmd/`, `internal/`), Dockerfile, `.env.example`
frontend/	SvelteKit application (`src/routes/`, `src/lib/`), Vitest config, Dockerfile, `.env.example`
proto/	Protobuf definitions (`hopper/pod/v1/`, `hopper/billing/v1/`) and `buf.yaml`
k8s/deploy/	Seven numbered raw YAML manifests (infra, apps, ingress, network policies, cert-manager, backup)
k8s/argocd/	ArgoCD app-of-apps root and child Application
charts/hopper/	Helm chart with `values.yaml`, `values-dev.yaml`, `values-staging.yaml`, `values-prod.yaml`
tests/	All test code: `unit/`, `integration/`, `orchestrator/` (Go), `e2e/` (Playwright), `load/` (k6), `security/`, `chaos/`, `mocks/`, `fixtures/`, `coverage/`
images/	VM container image Dockerfiles (`hopper-vm`, `-python`, `-cpp`, `-java`)
observability/	Prometheus config + alert rules, Grafana dashboard JSON, Loki config
infrastructure/	Ansible playbooks + inventory, Pulumi IaC skeleton
scripts/	`ci/` (docker-build), `cd/` (k8s-rollout), `test/` (run.sh), `dev-setup.sh`, `deploy-local.sh`, `generate-proto.sh`, `keycloak-harden.sh`
docs/	Architecture, SDD, SRS, testing plan, E2E guide, tech-stack, tasks
.github/workflows/	Five GitHub Actions workflow files
Makefile	Convenience targets mirroring `scripts/test/run.sh` commands
docker-compose.yml	Local infra (PostgreSQL/TimescaleDB, NATS, Keycloak)

Within `services/api-gateway/app/` the internal layout follows FastAPI conventions: `routers/` (one module per resource group), `services/` (business logic), `schemas/` (Pydantic models), `core/` (database, NATS, auth middleware), `middleware/`, and `proto/` (generated gRPC stubs). Within `services/orchestrator/internal/` the layout mirrors Go package conventions: `k8s/` (pod and network-policy management), `billing/` (ticker), `pod/` (manager), `grpc/` (server), and `config/`.

7.4 Local Setup Instructions

The following steps reproduce the local development environment. They are derived from `README.md`, `scripts/dev-setup.sh`, and the per-service `.env.example` files.

Prerequisites: Docker and Docker Compose, Go 1.23+, Node.js 22+, pnpm 10+ (or 9+), Python 3.12+, Poetry, kubectl with a k3s cluster (optional for orchestrator development).

1. **Clone and configure environment files.**

```
git clone https://github.com/CREVIOS/Hopper.git
cd Hopper
cp .env.example .env
cp services/api-gateway/.env.example services/api-gateway/.env
cp services/orchestrator/.env.example services/orchestrator/.env
cp frontend/.env.example frontend/.env
```

   Edit each `.env` as needed (defaults work for local dev without changes).

2. **Start infrastructure services.**

```
docker compose up -d   # PostgreSQL/TimescaleDB :5433, NATS :4222, Keycloak
    :8080
```

   Wait for PostgreSQL healthcheck: `docker compose ps`.

3. **Install dependencies and run migrations.**

```
# Frontend
cd frontend && pnpm install && cd ..
# API Gateway
cd services/api-gateway && poetry install
poetry run alembic upgrade head && cd ../..
# Orchestrator
cd services/orchestrator && go mod download && cd ../..
```

   Alternatively, `./scripts/dev-setup.sh` performs all of the above steps, including waiting for PostgreSQL readiness.

4. **Start application services** (three terminals):

```
# Terminal 1 - Frontend (http://127.0.0.1:5173)
cd frontend && pnpm dev

# Terminal 2 - API Gateway (http://127.0.0.1:8000)
cd services/api-gateway
poetry run uvicorn app.main:app --reload --port 8000
```

```
# Terminal 3 - Orchestrator (gRPC :50051)
cd services/orchestrator && go run ./cmd/orchestrator/
```

5. **Dev login.** Navigate to `http://127.0.0.1:5173` and use *Dev login (skip SSO)* with the credentials from `frontend/.env` (`DEV_LOGIN_USER`/`DEV_LOGIN_PASS`). For a full Keycloak SSO login flow, configure Keycloak as described in the *Deployment Steps* section.

6. **Run the test suite.**

```
make test-unit          # Python unit tests (pytest)
make test-frontend      # Vitest with coverage
make test-orchestrator  # Go orchestrator tests
make test-e2e           # Playwright mock-backed E2E
```

   The `make help` target lists all available test commands. All test commands delegate to `./scripts/test/run.sh`.

8. Evaluation & Reflection

8.1 Web Performance & Core Web Vitals

Google PageSpeed Insights, powered by Lighthouse, was used to evaluate the deployed Hopper landing page in desktop mode. Since Hopper is primarily intended for desktop-based compute management, the desktop audit is treated as the main performance evaluation. The desktop test achieved a performance score of 99, with strong results across all measured categories.

Table 15: Desktop Lighthouse audit results generated through Google PageSpeed Insights.

Category or Metric	**Desktop Result**
Performance score	99/100
Accessibility	96/100
Best Practices	100/100
SEO	100/100
Agentic Browsing	2/2
First Contentful Paint (FCP)	0.8 s
Largest Contentful Paint (LCP)	0.8 s
Total Blocking Time (TBT)	0 ms
Cumulative Layout Shift (CLS)	0.001
Speed Index	1.1 s

The results indicate that the deployed landing page loads quickly and remains visually stable. The 0 ms Total Blocking Time shows that JavaScript execution caused no measurable main-thread blocking during the audit. The CLS value of 0.001 indicates almost no unexpected layout movement, while the 0.8 s FCP and LCP values demonstrate fast initial and primary-content rendering. Figure 7 shows the category scores reported by the audit, and Figure 8 shows the individual metric measurements underlying the performance score.

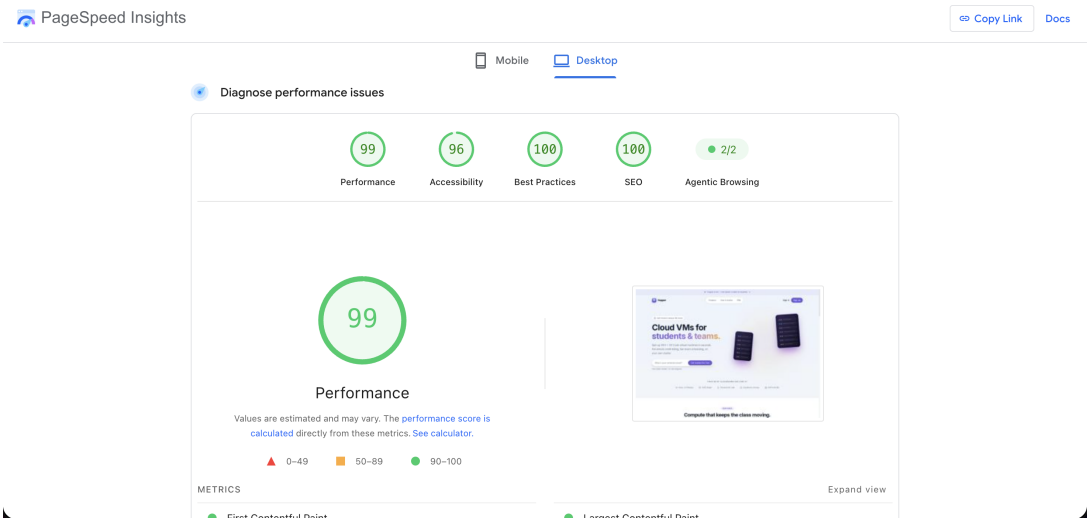

Figure 7: Google PageSpeed Insights category scores for the deployed Hopper landing page in desktop mode.

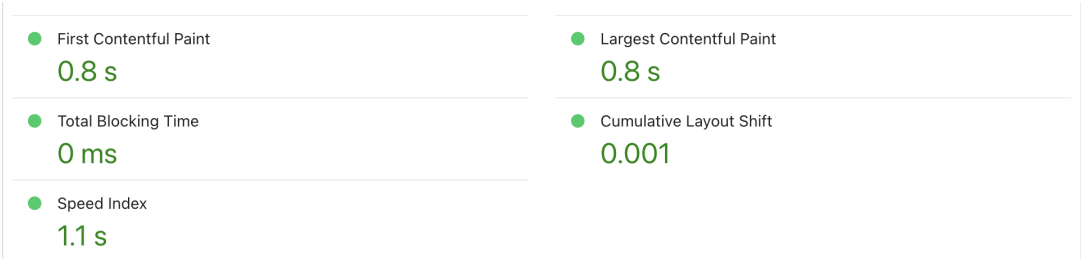

Figure 8: Core Web Vitals and supporting metric measurements from the same desktop audit.

8.2 Challenges & Solutions

Developing Hopper required coordinating several independently deployed components, including SvelteKit, FastAPI, Go, PostgreSQL, NATS, Keycloak, and Kubernetes. Differences in communication protocols, configuration, startup order, and failure behaviour made local development and deployment difficult. This was addressed by defining clear REST, gRPC, and event-based communication paths, providing Docker Compose services for local infrastructure, and validating the complete system through automated CI workflows.

Authentication was another major challenge because Hopper supports OIDC login, direct email-password login, email verification, token refresh, logout, API keys, and three application roles. Synchronisation between Keycloak identity data, application cookies, frontend route guards, and local user records created several edge cases. The solution was to centralise token validation in the API gateway, use JWKS-based JWT verification, store tokens in secure HttpOnly cookies, implement silent refresh, and enforce roles and resource ownership independently in backend routes.

Resource provisioning also required careful handling of limited CPU and memory. Multiple users may request environments simultaneously, while provisioning must respect cluster capacity and available credits. Hopper addresses this through a persistent admission queue, capacity reservation, scheduler leader election, plan-based resource limits, and credit checks before provisioning. Billing operations were made idempotent so that repeated NATS events do not deduct credits multiple times, while grace-period handling allows sessions to be terminated safely after credit exhaustion.

Real-time monitoring and interactive access introduced additional complexity because the platform uses REST requests, Server-Sent Events, WebSockets, SSH forwarding, and code-server proxying. These channels were separated by responsibility and routed through the API gateway, where authentication and pod ownership are checked before traffic is forwarded. Layered unit, integration, end-to-end, load, security, and orchestration tests were then added to identify failures across these boundaries.

8.3 Limitations & Future Work

The current production deployment uses a single-node k3s cluster. Although this is sufficient for demonstration and small institutional deployments, it provides limited horizontal scalability and creates a single point of failure at the cluster level. Future work should restore and stabilise multi-node worker support, distribute workloads across multiple hosts, and validate recovery when nodes or control-plane services become unavailable.

The admission scheduler currently follows a first-come, first-served head-of-line policy. A large request at the front of the queue may therefore delay smaller requests that could otherwise be admitted. Future versions could support safe backfilling, priorities, quotas, estimated waiting times, and fairness policies based on courses, roles, or historical resource consumption.

Several infrastructure and security mechanisms also require further improvement. Rate limits are stored in process memory and are not shared consistently across multiple gateway replicas. This should be replaced with Redis-backed or ingress-level distributed rate limiting. File uploads should enforce explicit size limits, filename validation, and content restrictions. Automated SSH connections should use stronger host verification, and the full end-to-end suite should eventually run against a real staging deployment containing Keycloak, NATS, PostgreSQL, the orchestrator, and Kubernetes rather than depending primarily on mocked services.

Some application areas remain incomplete or intentionally lightweight. The settings endpoint does not yet provide full persistent configuration, the backend has no dedicated repository abstraction, and some chaos and security checks require manual staging execution. Future work should improve persistence boundaries, expand automated full-stack testing, complete GitOps-based deployment, strengthen backup restoration testing, and broaden dashboards and alerting for non-GPU and application-level metrics.

A planned extension is an AI-based user assistance agent integrated into the Hopper dashboard. The agent could guide users through selecting plans, creating environments, understanding credit usage, resolving common provisioning errors, locating documentation, and interpreting resource metrics. The first version should remain advisory and read-only. Any later capability to perform actions should use authenticated Hopper APIs, require explicit user confirmation, respect role permissions, and produce auditable action logs.

8.4 Lessons Learned

The project demonstrated that a distributed web platform requires clear service boundaries and communication contracts. Separating the browser-facing frontend, API gateway, orchestration service, event bus, identity provider, and cluster runtime made the system easier to extend, but also showed that every boundary introduces configuration, failure-handling, and testing responsibilities.

Another important lesson was that authorization must be enforced by the backend rather than relying on hidden frontend pages. Route guards improve user experience, but pod ownership, role restrictions, credit allocation, file access, terminal access, and administrative operations must all be verified again by the API.

The event-driven parts of Hopper showed the importance of idempotency, reconciliation, and persistent state. NATS improves decoupling between billing, metrics, notifications, and lifecycle events, but consumers must remain correct when messages are delayed, repeated, or processed after a service restart.

Testing proved most valuable when it was introduced across multiple layers. Unit tests found logic errors quickly, integration tests exposed database, migration, Keycloak, and NATS configuration problems, and end-to-end and load tests revealed issues that were not visible from individual modules. CI/CD, infrastructure validation, and documentation therefore became part of the implementation rather than activities performed only after development.

Finally, the project showed that usability is as important as infrastructure correctness. Users should not need detailed knowledge of Kubernetes, ports, proxies, or billing events to create and manage a compute environment. Future development, including the planned AI assistant, should continue reducing this operational complexity while preserving security, transparency, and user control.

8.5 Individual Responsibility

Development responsibilities were distributed according to each member's primary area of ownership. The estimated contribution percentages are based on the documented distribution of 173 commits and are used only as an approximate indicator, since commit count alone does not fully measure implementation effort.

Table 16: Individual responsibilities and contribution summary

Member Name	**Core Responsibilities**	**Key Contributions**
Tazkia Malik	Testing and quality assurance	Built the unit, integration, end-to-end, frontend, Go orchestrator, load, chaos, and security tests; maintained coverage reporting and resolved test-related infrastructure and migration issues.
Anindya Kundu	Authentication, security, operations, notifications, and CI/CD	Implemented the web terminal and SSH proxy, authentication and verification flows, RBAC, rate limiting, network groups, secrets scanning, reliability features, Helm and ArgoCD configuration, and real-time notifications.
Muhaiminul Islam Ninad	Infrastructure, deployment pipelines, and documentation	Added Kubernetes support, improved JWT validation and billing handling, created image-publishing and SSH-deployment workflows, integrated Docker and Makefile automation, and prepared the SRS and SDD documents.
Sadek Hossain Asif	Project foundation, backend core, and orchestration	Founded and scaffolded the project; developed the initial authentication, pod, credit, migration, orchestration, metrics, billing, SSH-key, audit-log, file-browser, and deployment functionality.
Kabya Mithun Saha	VM admission control and capacity management	Designed the CPU and memory admission queue, live resource-availability calculation, and capacity-accounting model; contributed to terminal integration, authentication UX, and the multi-node clustering experiment.

Member Name	Core Responsibilities	Key Contributions
Tanzila Khan	Frontend and user-interface development	Developed the initial cloud-platform interface, reusable component and table libraries, pagination, credits and login pages, SSH-key management, file browser, charts, administrative dashboards, issue reporting, and general UI improvements.
Taif Ahmed Turjo	Billing, API security, and hardening	Implemented durable billing and prorated final charges, ownership enforcement, Kubernetes secret handling, API RBAC, admin ingress protection, saved SSH-key installation, credit-grace notifications, and CI audit checks.
Fayek Ahmed	Code-server integration	Implemented the code-server integration used to provide browser-accessible development environments.

A. Screenshots / UI Walkthrough

The screenshots below were captured from the live deployment at https://hopper.farefin.com at a viewport of 1440×900 with a $2\times$ device pixel ratio. Thirteen screenshots are included. Screens 1–7 and 10–12 use an authenticated `Student` account; Screen 8 uses an `Admin` account authenticated through the University SSO (Keycloak OIDC) flow; and Screen 9 uses a `Teacher` account, each console being reachable only by its corresponding role.

The landing page is roughly three screens tall, so it is reproduced in two halves.

Remark on role gating. The `/admin` and `/teacher` routes are gated independently rather than by a single privilege hierarchy, and this was verified from three directions. A `Student` account is redirected to `/dashboard` from both routes; the `Admin` account reaches `/admin` but is *also* redirected away from `/teacher`; and the `Teacher` account reaches `/teacher`. Each console is therefore gated on its own specific role, so elevation to administrator does not confer access to the professor console.

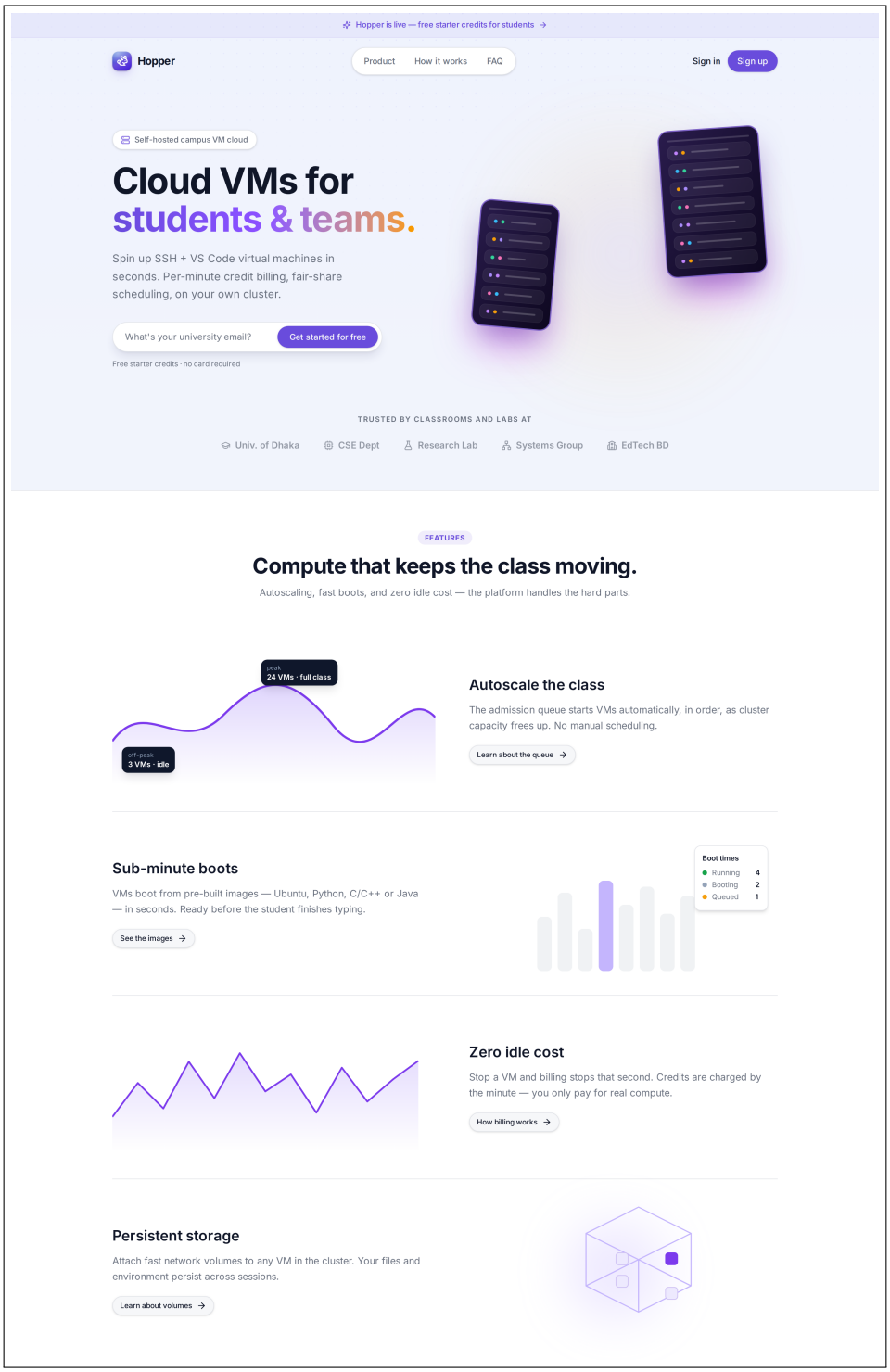

Figure 9: Screen 1a: Public Landing Page — Upper Half (/) — Navbar and hero section with the Hopper wordmark, tagline, and primary call-to-action, followed by the feature highlights. Captured logged out.

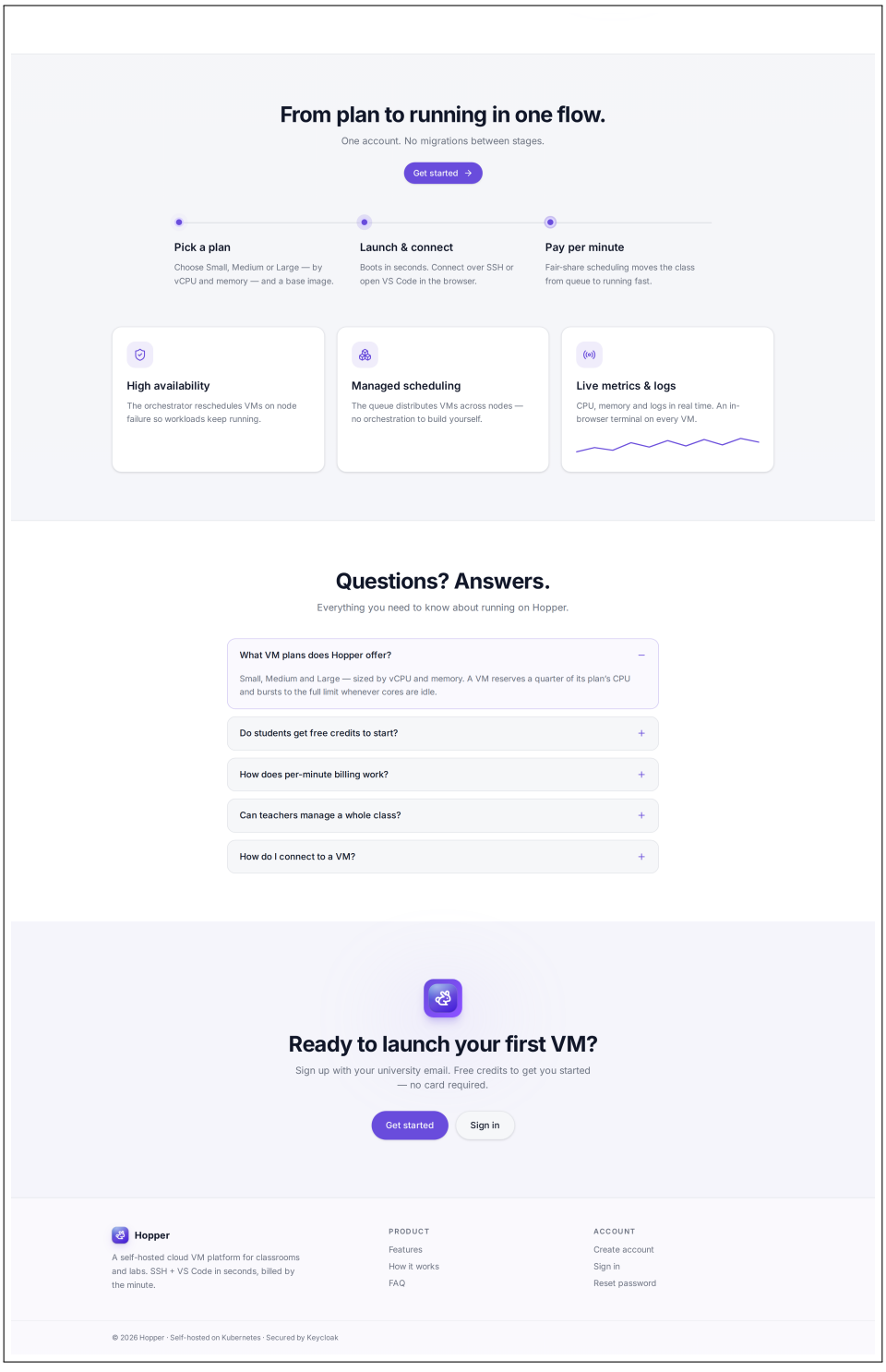

Figure 10: Screen 1b: Public Landing Page — Lower Half (/) — Continuation of the same page: VM plan cards (Small / Medium / Large) with their per-hour credit rates, the FAQ section, and the page footer.

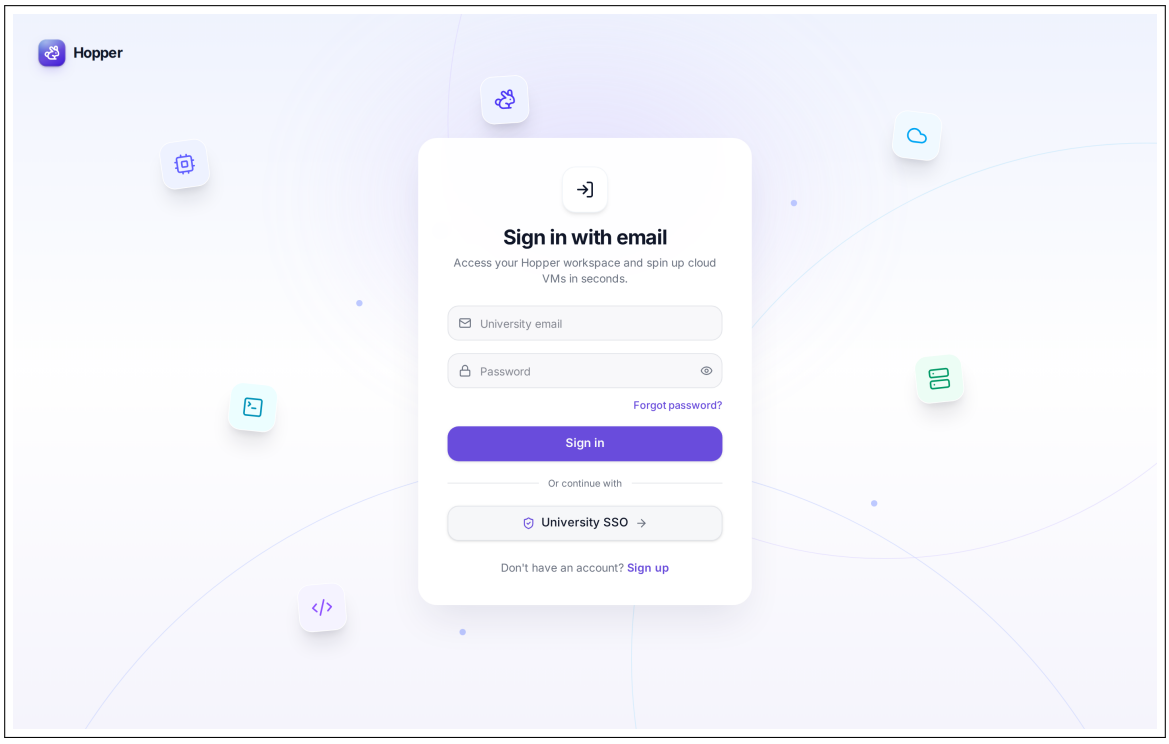

Figure 11: Screen 2: Login (`/login`) — "Sign in with email" heading; email and password fields; "Forgot password?" link; "University SSO" button for the Keycloak OIDC flow; link to the signup page.

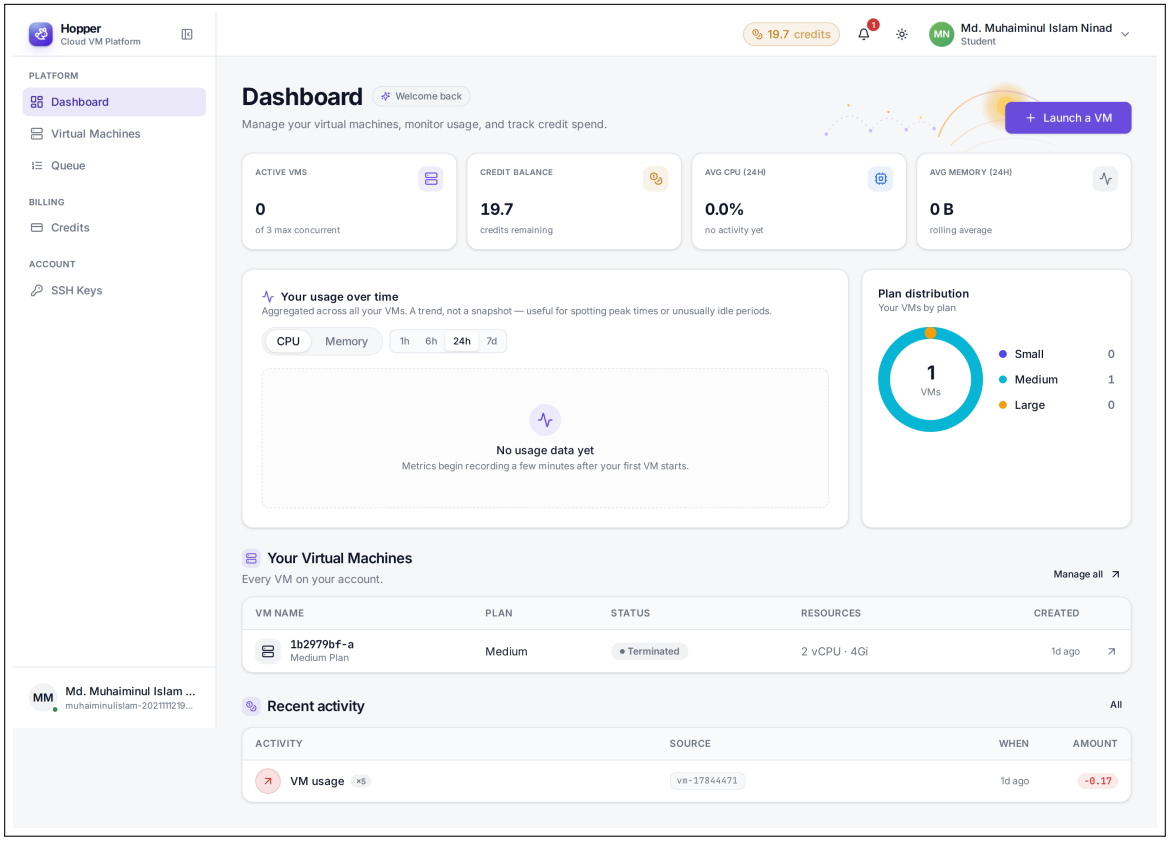

Figure 12: Screen 3: Student Dashboard (`/dashboard`) — Stat cards for active VMs, credit balance, and 24-hour average CPU and memory. "Your usage over time" chart with CPU/Memory and 1h/6h/24h/7d selectors, plan-distribution donut, the user's VM table, and recent credit activity.

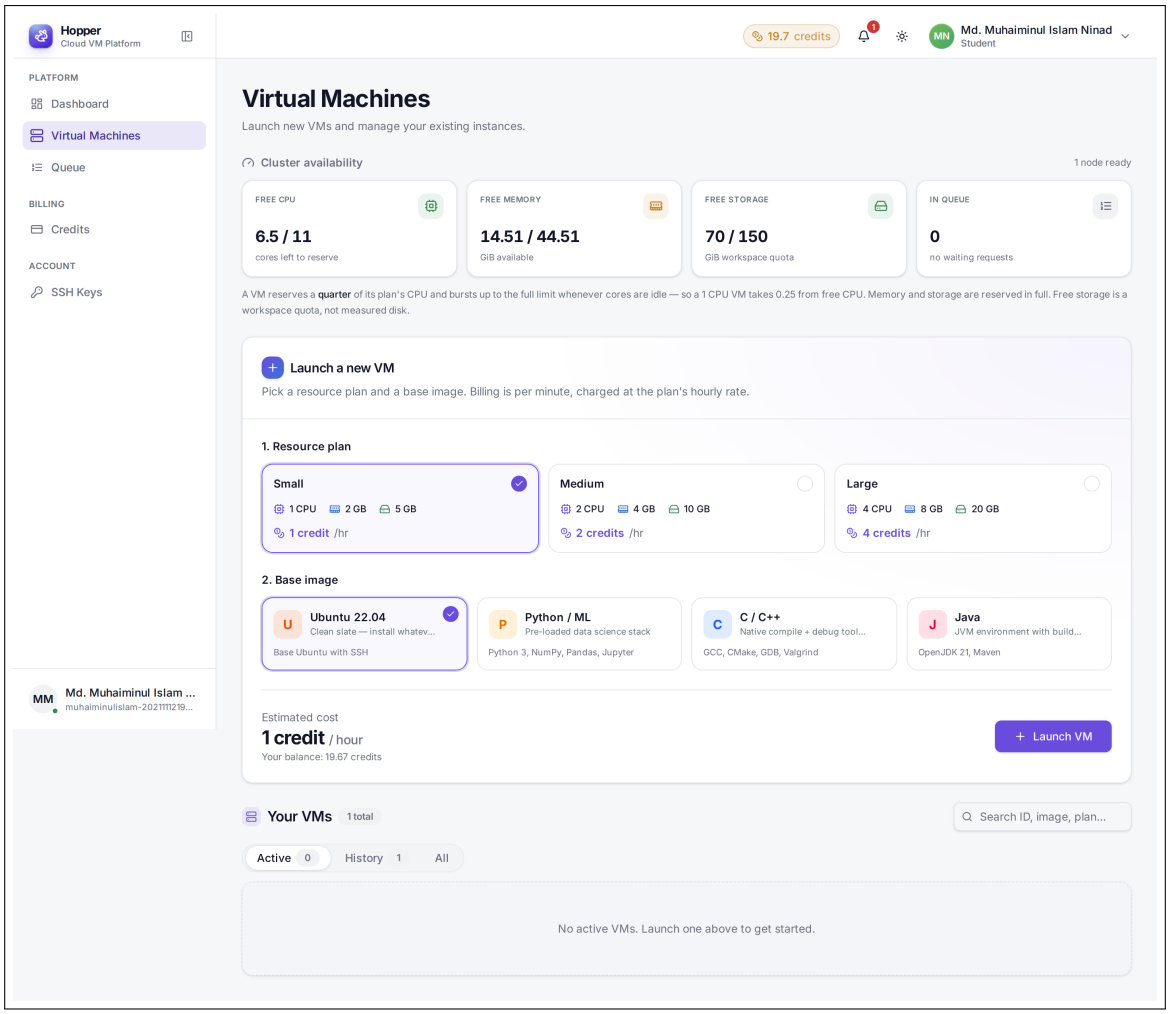

Figure 13: Screen 4: VM List & Launch (`/pods`) — Cluster-availability cards (free CPU, memory, storage, queue depth), the launch form with its resource-plan selector (Small / Medium / Large) and base-image selector (Ubuntu, Python/ML, C/C++, Java), estimated cost, and the user's VM list with Active/History/All filters.

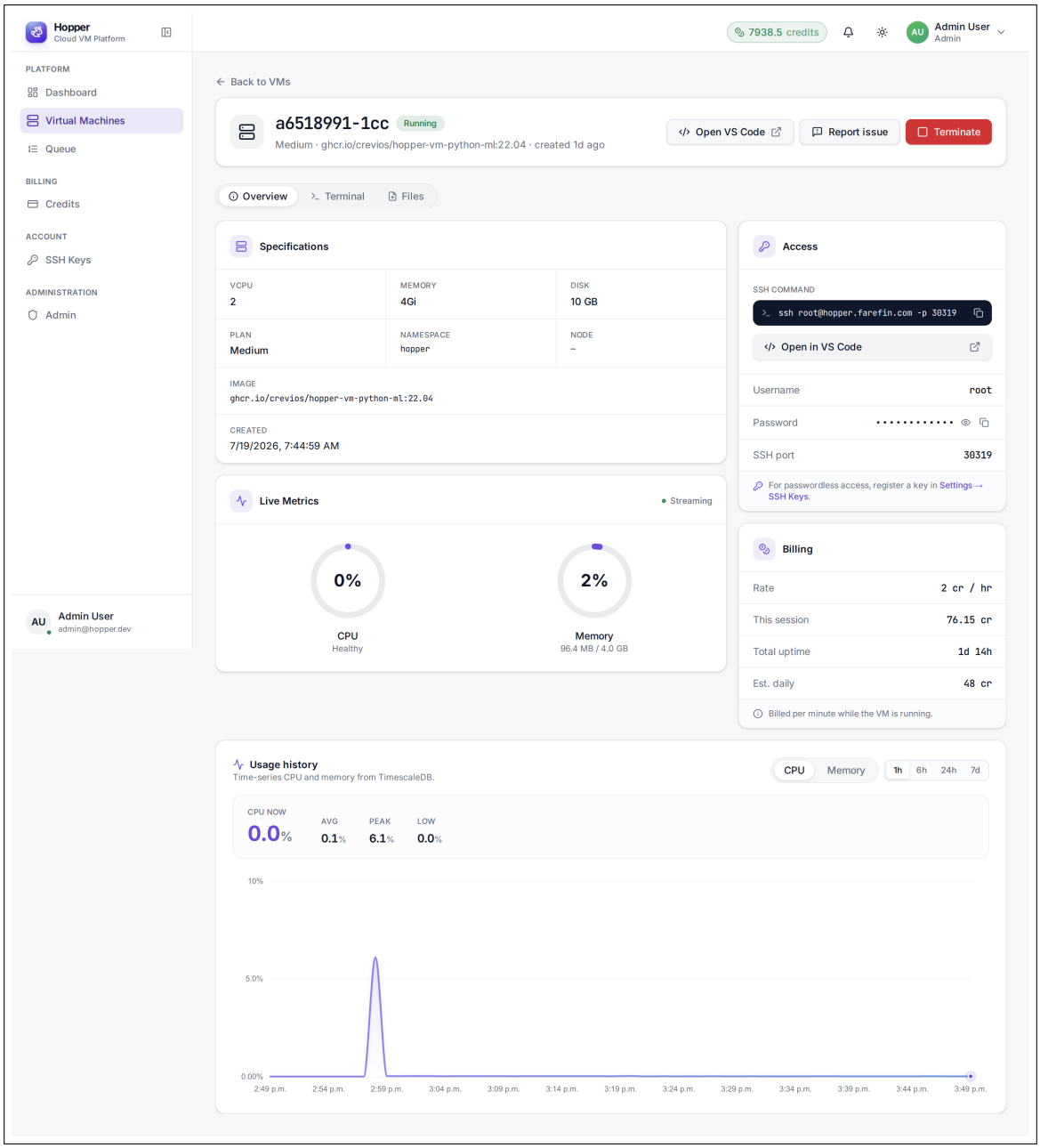

Figure 14: Screen 5: VM Detail — Overview Tab (`/pods/[id]`) — VM header with ID and status badge; Overview / Terminal / Files tabs; Specifications panel (vCPU, memory, disk, plan, namespace, node, image); Access panel showing the generated SSH command, port, and masked password, plus an "Open in VS Code" launcher; Live Metrics panel streaming CPU and memory over SSE (note the *Streaming* indicator); Billing panel with rate, accrued session cost, and total uptime; and the TimescaleDB-backed usage-history chart with CPU/Memory and 1h/6h/24h/7d selectors. Captured against a running VM.

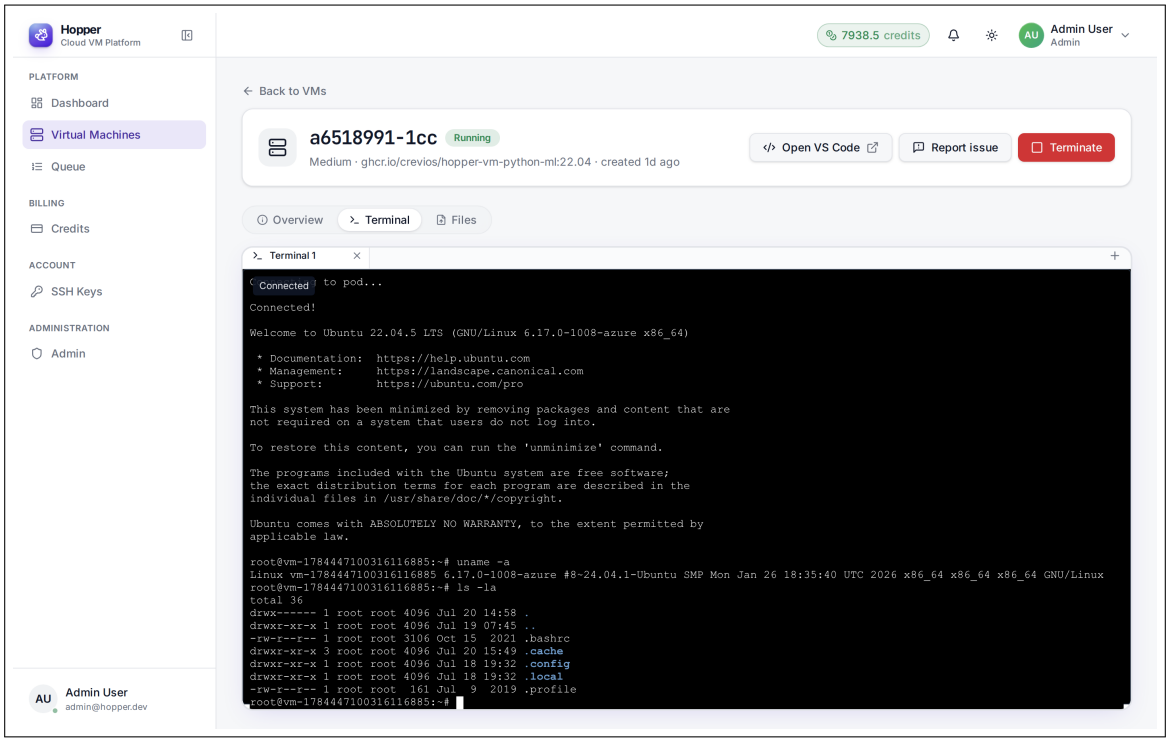

Figure 15: Screen 6: VM Detail — Terminal Tab (`/pods/[id]?tab=terminal`) — Full-height xterm.js terminal attached to the VM over a WebSocket, showing the session tab bar with its *Connected* status badge and a "new terminal" control. The transcript shows the Ubuntu 22.04.5 login banner followed by `uname -a` and `ls -la` executed inside the container, confirming a live interactive shell rather than a rendered mock-up.

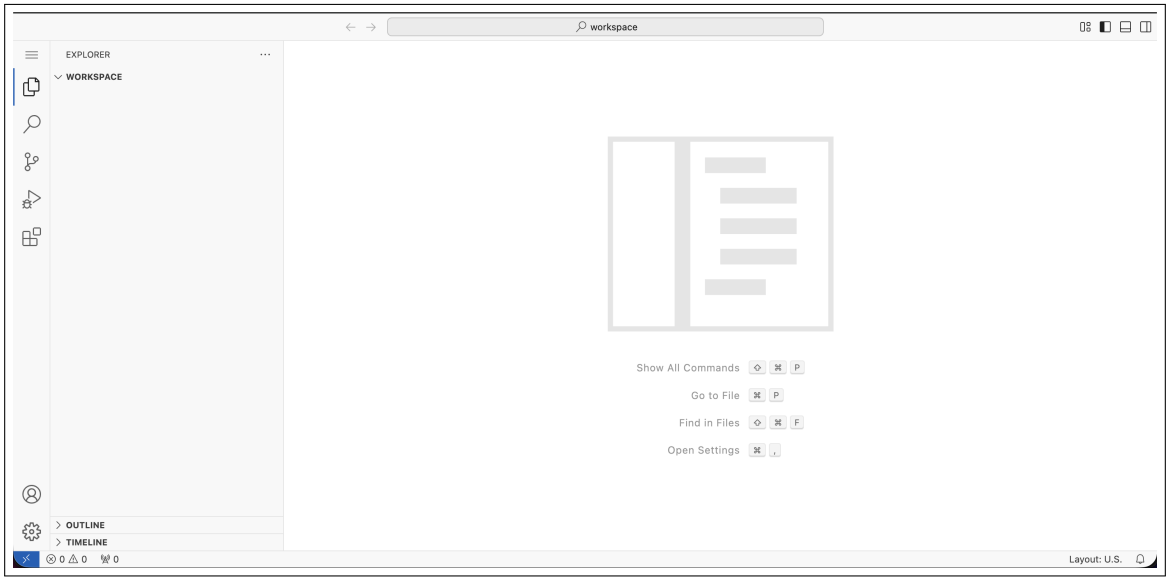

Figure 16: Screen 6b: Browser-Based VS Code Workspace — The code-server workspace opened from the VM detail page through the authenticated VS Code proxy, showing the Explorer sidebar, an active editor, and the integrated terminal inside the same browser session. This confirms that Hopper provides a full in-browser development environment in addition to the standalone terminal tab.

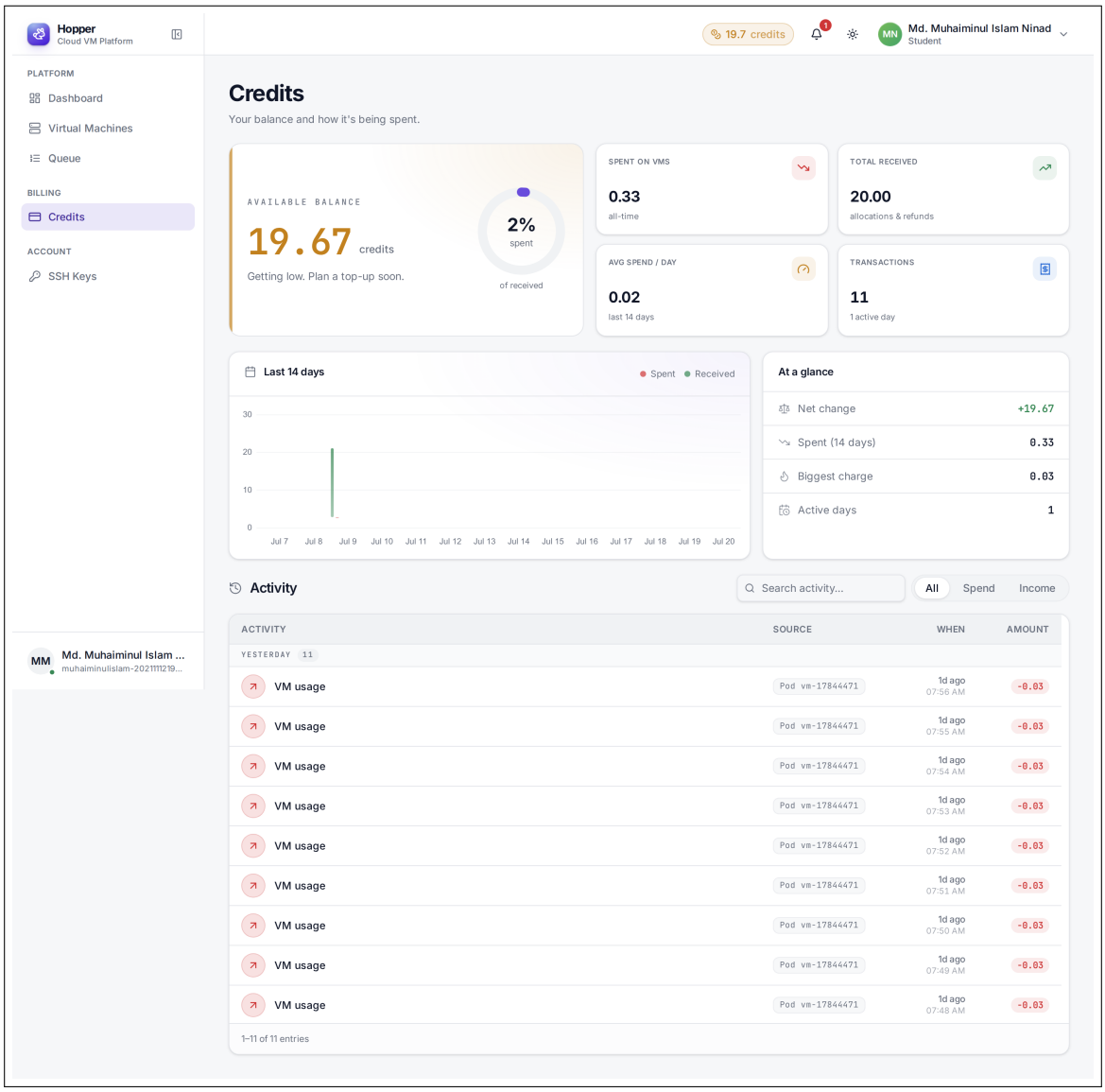

Figure 17: Screen 7: Credits (`/credits`) — Credit-balance display and the transaction-history table listing timestamp, description, source VM, and debit/credit amount.

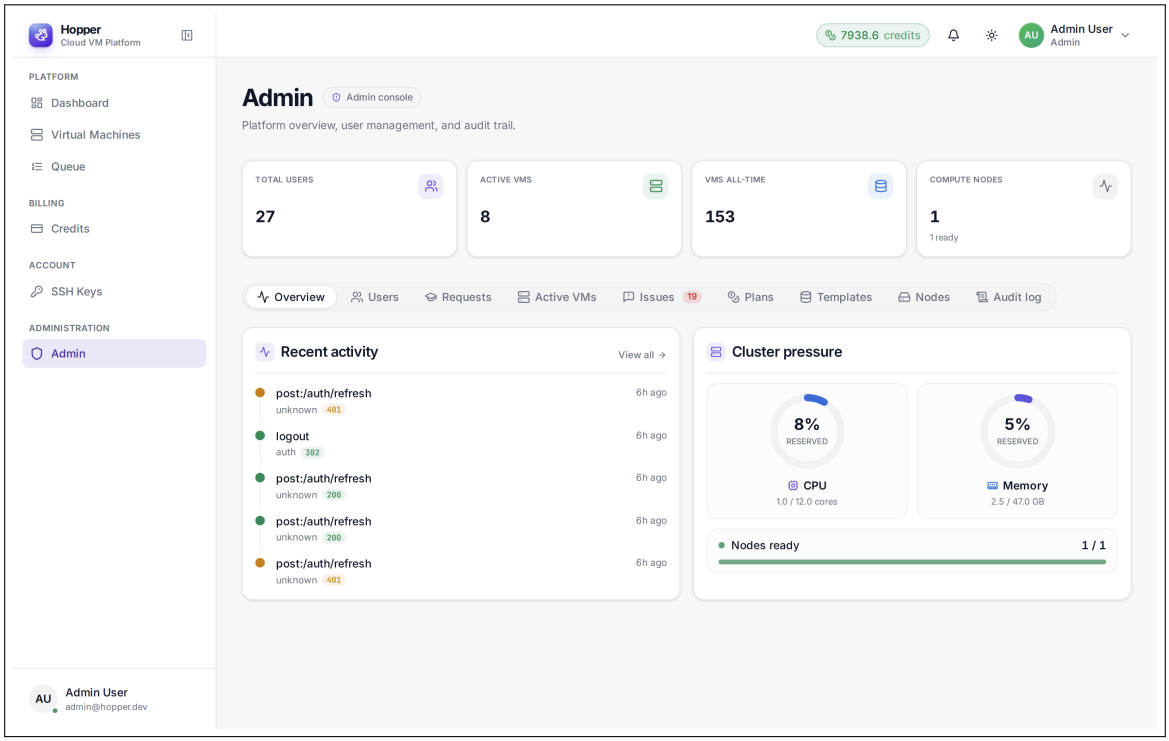

Figure 18: Screen 8: Admin Console — Overview (`/admin`) — Platform stat cards (total users, active VMs, all-time VMs, compute nodes) and the full tab bar: Overview, Users, Requests, Active VMs, Issues, Plans, Templates, Nodes, and Audit log. The Overview tab pairs a recent activity feed — API route, outcome status code, and relative timestamp — with a cluster-pressure panel showing reserved CPU and memory against capacity and node readiness.

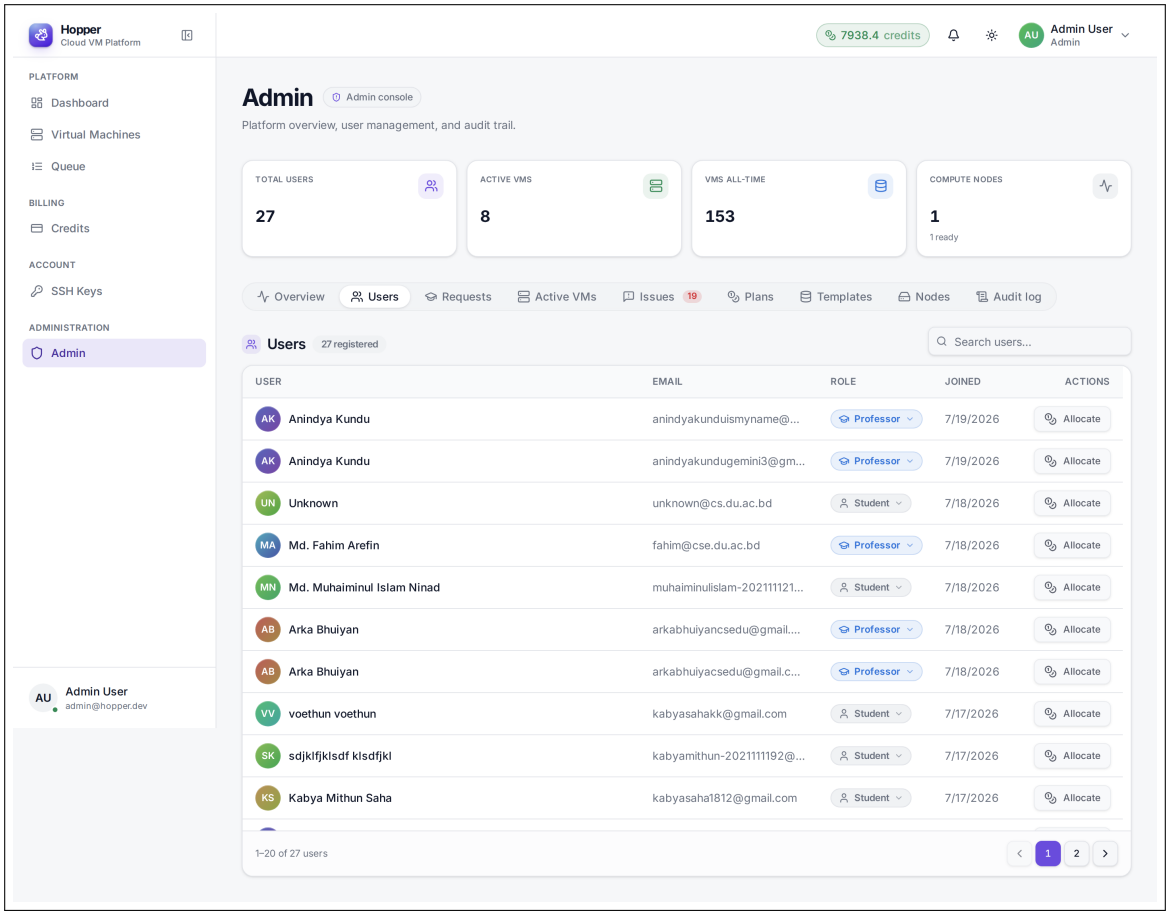

Figure 19: Screen 8b: Admin Console — Users Tab — Registered-user table with avatar, name, email, role badge, join date, and per-row administrative actions, backing the role-management and credit-allocation flows.

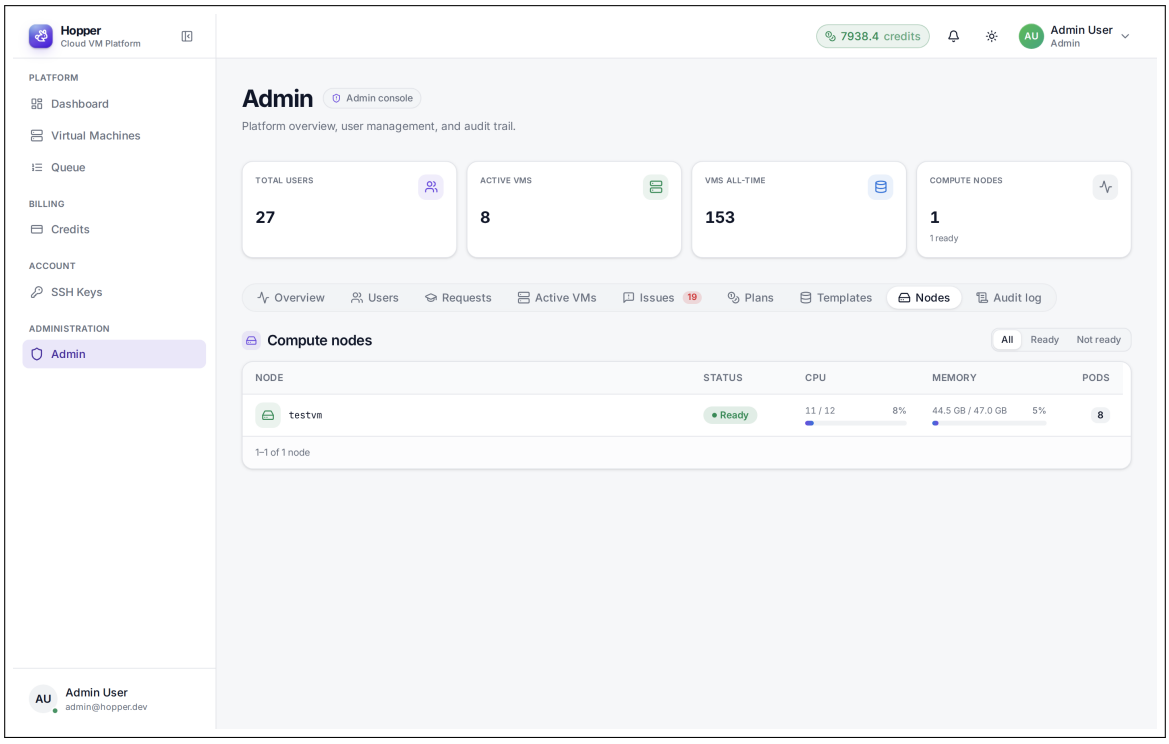

Figure 20: Screen 8c: Admin Console — Nodes Tab — Kubernetes node inventory with All / Ready / Not-ready filters, listing each node's status and its CPU, memory, and pod capacity.

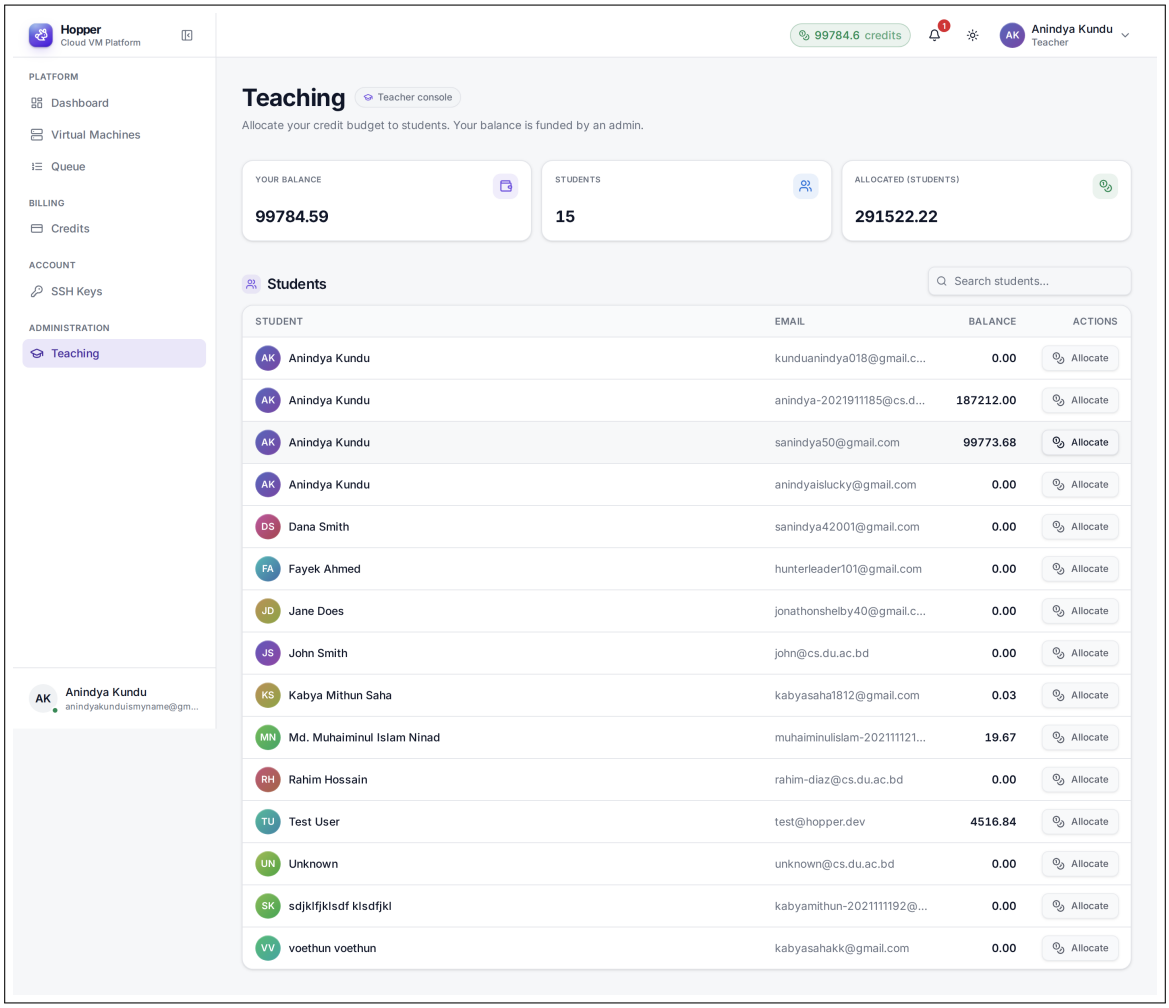

Figure 21: Screen 9: Teacher (Professor) Console (`/teacher`) — Summary cards for the professor's own credit balance, enrolled student count, and total credits already allocated. Beneath them, the searchable student roster lists each student's name, email, and current balance with a per-row "Allocate" action, implementing the delegated credit-distribution model in which an administrator funds a professor, who in turn funds students.

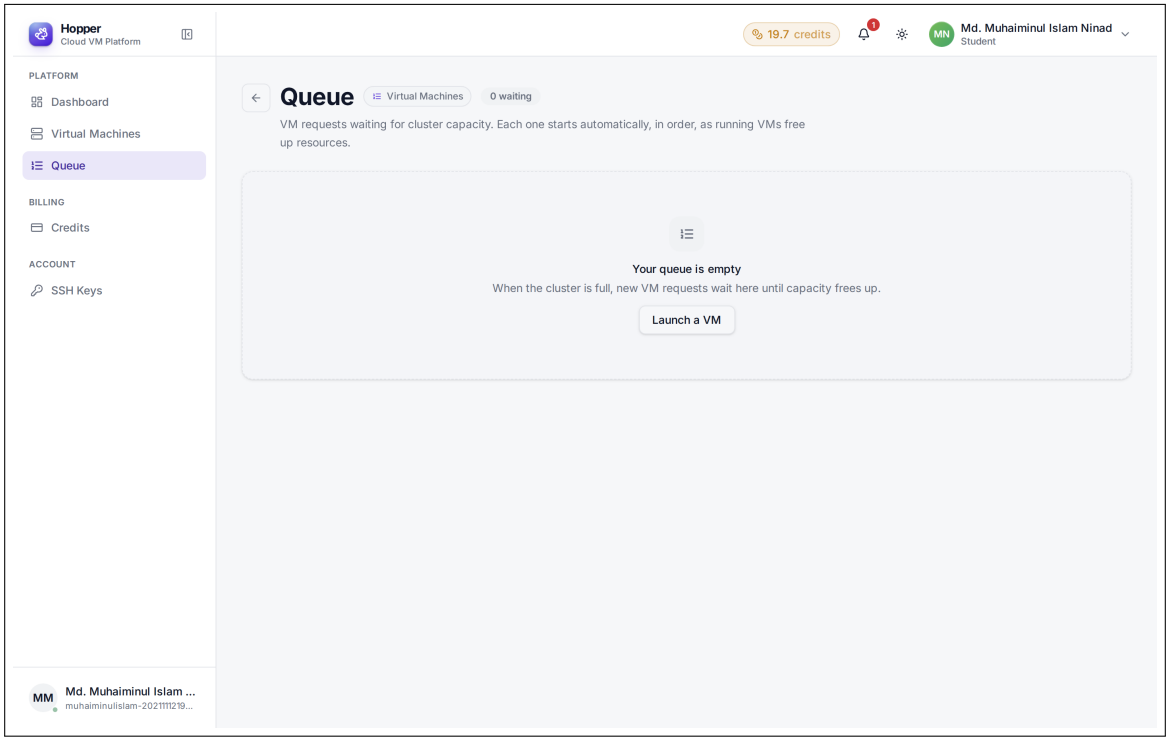

Figure 22: Screen 10: VM Queue (`/pods/queue`) — Queue view, shown in its empty state — the cluster had spare capacity at capture time, so no requests were waiting. When capacity is exhausted, this screen lists each queued request with its position, requested plan, and a cancel control.

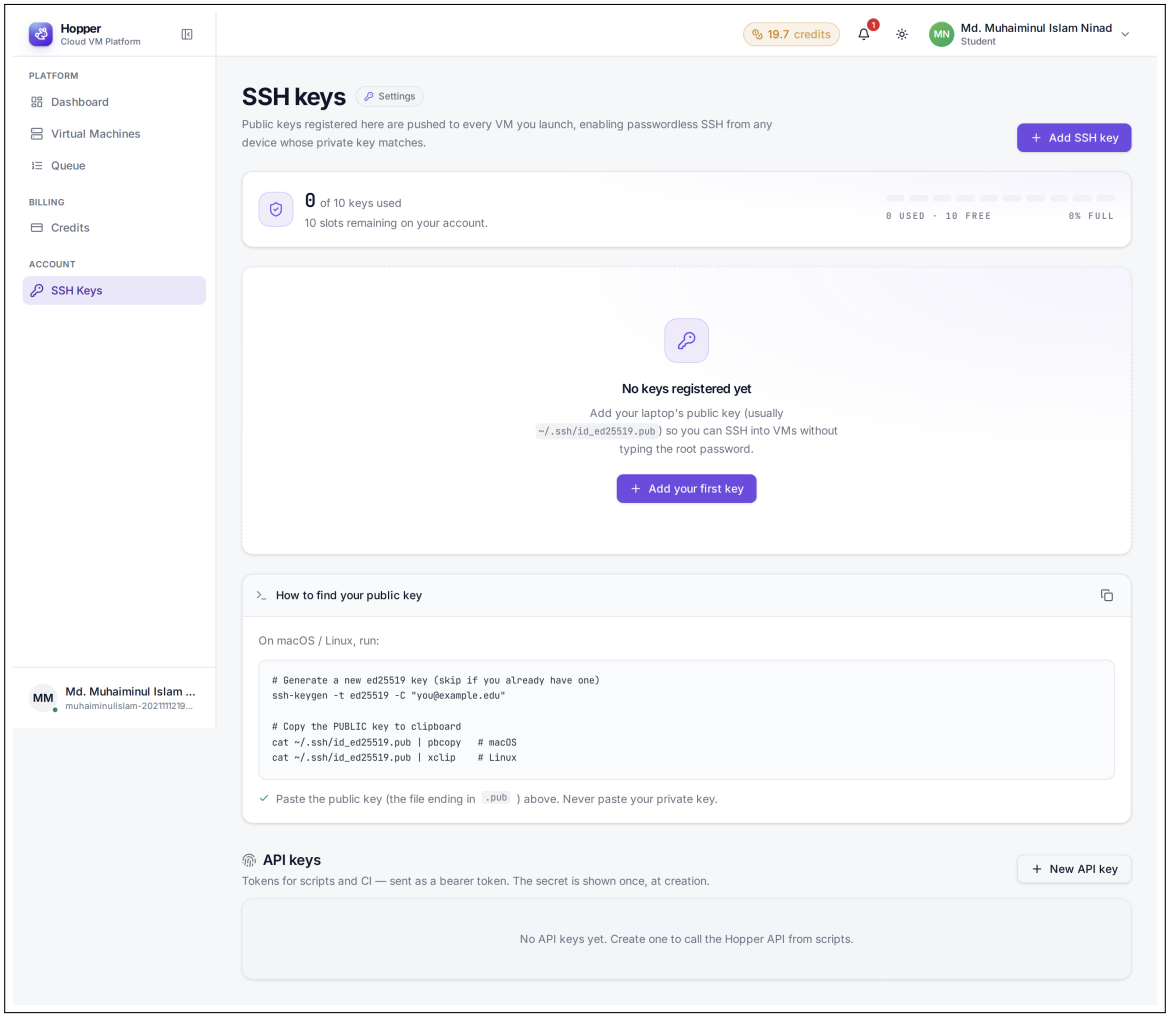

Figure 23: Screen 11: Settings — SSH Keys (`/settings/ssh-keys`) — Registered SSH public keys with name, fingerprint, and creation date; the "Add SSH key" form; and per-key deletion controls.

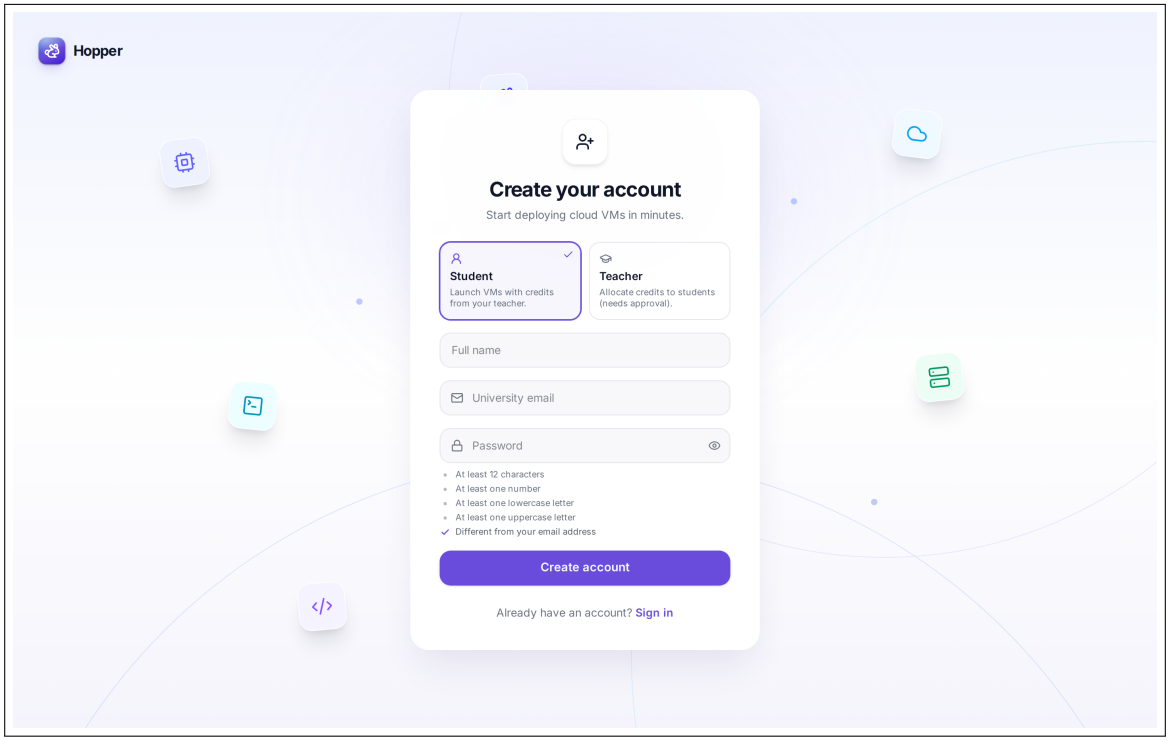

Figure 24: Screen 12: Signup (`/signup`) — Registration form with name, email, and password fields and the password-policy feedback. The subsequent verification-code screen (`/verify-email`) is reachable only by completing a registration and was therefore not captured.